



COMPILED PROJECT PROPOSALS

Three Machine Learning Project Proposals

Compiled Machine-Learning Project Proposals for Final-Year Selection

1. Crowd Counting • 2. Handwritten Text Recognition • 3. Sign Language Recognition

Riphah International University

Department of RISE MSAI

Course / Subject: Machine Learning

Submitted By — Group Members	Submitted To
1. Azhar Hussain <i>(Group Lead)</i> 2. Rahma 3. Haris Javaid 4. Syed Farakh Abbas	Junaid Khan <i>Course Instructor — Machine Learning</i>

Date of Submission: 11 May 2026

Academic Session 2025-2026 | Confidential — For Academic Evaluation Only

Master Table of Contents

If the entries below do not display page numbers in your viewer, place the cursor in the table and press Ctrl+A then F9 (Word) — “Update Field → Update entire table”.

0. About This Document.....	6
1.0. Contents — Proposal 1: AI-Based Crowd Counting System.....	8
2. Abstract.....	8
3. Introduction.....	8
4. Problem Statement.....	9
5. Objectives.....	9
6. Scope of the Project.....	10
In Scope.....	10
Out of Scope (Limitations).....	10
7. Literature Review.....	10
7.1 Foundational and Recent Works.....	10
7.2 Comparison of Crowd-Counting Approaches.....	11
7.3 Benchmark Datasets.....	11
8. Proposed Methodology.....	12
8.1 System Architecture.....	12
8.2 Step-by-Step Workflow.....	12
8.3 Workflow Flowchart.....	13
8.4 Algorithms, Models & Frameworks.....	14
9. Technologies and Tools.....	14
10. Dataset Information.....	14
10.1 Candidate Datasets.....	14
10.2 Data Pre-processing.....	15
10.3 Training / Validation / Testing Split.....	15
10.4 Data Augmentation.....	15
11. Expected Outcomes.....	15
12. Innovation / Uniqueness.....	16
13. Feasibility Analysis.....	16
13.1 Technical Feasibility.....	16
13.2 Cost Analysis.....	17
13.3 Hardware Feasibility & GPU / Resource Limitations.....	17
13.4 Practical Implementation Plan.....	17
14. Hardware and Software Requirements.....	17
15. Future Enhancements.....	18

16. Timeline / Gantt Chart.....	18
17. Conclusion.....	20
18. References.....	20
2.0. Contents — Proposal 2: AI-Based Handwritten Text Recognition System.....	22
2. Abstract.....	22
3. Introduction.....	22
4. Problem Statement.....	23
5. Objectives.....	23
6. Scope of the Project.....	24
In Scope.....	24
Out of Scope (Limitations).....	24
7. Literature Review.....	24
7.1 Key Works.....	24
7.2 Comparison of HTR Approaches.....	25
7.3 Datasets & Resources.....	25
8. Proposed Methodology.....	26
8.1 System Architecture.....	26
8.2 Step-by-Step Workflow.....	26
8.3 Workflow Flowchart.....	28
8.4 Algorithms, Models & Frameworks.....	28
9. Technologies and Tools.....	29
10. Dataset Information.....	29
10.1 Candidate Datasets.....	29
10.2 Data Pre-processing.....	30
10.3 Training / Validation / Testing Split.....	30
10.4 Data Augmentation.....	30
11. Expected Outcomes.....	30
12. Innovation / Uniqueness.....	31
13. Feasibility Analysis.....	31
13.1 Technical Feasibility.....	31
13.2 Cost Analysis.....	31
13.3 Hardware Feasibility & GPU / Resource Limitations.....	32
13.4 Practical Implementation Plan.....	32
14. Hardware and Software Requirements.....	32
15. Future Enhancements.....	33
16. Timeline / Gantt Chart.....	33

17. Conclusion.....	34
18. References.....	34
3.0. Contents — Proposal 3: Sign Language Recognition System.....	37
2. Abstract.....	37
3. Introduction.....	37
4. Problem Statement.....	38
5. Objectives.....	38
6. Scope of the Project.....	39
In Scope.....	39
Out of Scope (Limitations).....	39
7. Literature Review.....	39
7.1 Key Works.....	39
7.2 Comparison of SLR Approaches.....	40
7.3 Datasets & Resources.....	40
8. Proposed Methodology.....	41
8.1 System Architecture.....	41
8.2 Step-by-Step Workflow.....	41
8.3 Workflow Flowchart.....	43
8.4 Algorithms, Models & Frameworks.....	43
9. Technologies and Tools.....	44
10. Dataset Information.....	44
10.1 Candidate Datasets.....	44
10.2 Data Pre-processing.....	45
10.3 Training / Validation / Testing Split.....	45
10.4 Data Augmentation.....	45
11. Expected Outcomes.....	45
12. Innovation / Uniqueness.....	46
13. Feasibility Analysis.....	46
13.1 Technical Feasibility.....	46
13.2 Cost Analysis.....	47
13.3 Hardware Feasibility & GPU / Resource Limitations.....	47
13.4 Practical Implementation Plan.....	47
14. Hardware and Software Requirements.....	47
15. Future Enhancements.....	48
16. Timeline / Gantt Chart.....	48
17. Conclusion.....	50

18. References.....50

0. About This Document

This master document compiles three candidate AI/ML project proposals prepared by the group for the Machine Learning course at Riphah International University (RISE MSAI). Each proposal is self-contained and follows the same academic structure — cover page, abstract, introduction, problem statement, objectives, scope, literature review, methodology, technologies, dataset information, expected outcomes, innovation, feasibility, requirements, future enhancements, timeline, conclusion and references. All three projects are deliberately scoped to be implementable by a four-member student team within one semester, using lightweight models, transfer learning, pretrained components and free cloud GPUs — without high-end hardware.

#	Project Title	Core Technique	Key Datasets	Headline Target
1	AI-Based Crowd Counting System	CSRNet density-map regression (transfer learning, frozen VGG-16)	ShanghaiTech, Mall	MAE $\approx$ 70-95 (SH-A); real-time dashboard
2	AI-Based Handwritten Text Recognition	CRNN (CNN + BiLSTM + CTC) + SymSpell post-correction	IAM + synthetic handwriting	CER $\approx$ 8-12 % on IAM; TXT/DOCX/PDF export
3	Sign Language Recognition System	MediaPipe 21-landmarks + MLP/1-D CNN (static) & LSTM/GRU (dynamic)	ASL Alphabet, Sign-MNIST, WLASL subset, self-collected	$\geq$ 95 % alphabet accuracy; real-time CPU + TTS

Table 0.1: Summary of the three proposed projects in this document.

The detailed proposals follow, beginning on the next page.



FINAL PROJECT PROPOSAL

AI-Based Crowd Counting System

Proposal 1 of 3 — A Final-Year Machine-Learning Project Proposal

Riphah International University

Department of RISE MSAI

Course / Subject: Machine Learning

Submitted By — Group Members	Submitted To
1. Azhar Hussain <i>(Group Lead)</i> 2. Rahma 3. Haris Javaid 4. Syed Farakh Abbas	Junaid Khan <i>Course Instructor — Machine Learning</i>

Date of Submission: 11 May 2026

Academic Session 2025-2026 | Confidential — For Academic Evaluation Only

1.0. Contents — Proposal 1: AI-Based Crowd Counting System

This proposal contains the standard sections (Abstract through References) as listed in the Master Table of Contents at the front of this document.

2. Abstract

Crowd management has become a critical concern for public safety at religious gatherings, transport hubs, stadiums, shopping malls and political rallies. Manual head-counting from closed-circuit television (CCTV) is slow, error-prone and impractical in dense scenes where individuals heavily occlude one another. This project proposes an AI-Based Crowd Counting System that estimates the number of people and the spatial density of a crowd directly from CCTV frames using a lightweight density-map regression network. The core model is a transfer-learning variant of CSRNet — a VGG-16 front-end (frozen, pretrained on ImageNet) followed by a small dilated convolutional back-end that outputs a per-pixel density map whose integral equals the estimated head count. Around this model the system adds a colour heatmap overlay, a threshold-based over-crowding alert engine and a real-time analytics dashboard built with Streamlit. The model is trained on publicly available datasets (ShanghaiTech Part A/B and the Mall dataset) using a free Google Colab GPU, making the entire pipeline reproducible by university students without high-end hardware. Expected performance is a Mean Absolute Error (MAE) in the region of 70–95 on ShanghaiTech Part A and below 15 on the sparser Mall dataset, with near real-time inference ($\approx$ 8–15 frames per second) on a mid-range laptop GPU or Colab. The deliverable is a deployable prototype dashboard plus a documented, retrainable model.



Keywords: Crowd counting, density estimation, CSRNet, dilated convolution, transfer learning, computer vision, CCTV analytics, public-safety AI, Streamlit dashboard.

3. Introduction

The volume of video data generated by surveillance infrastructure has grown faster than the human capacity to analyse it. In Pakistan alone, large-scale events — Friday congregations at major mosques, Eid gatherings, Muharram processions, cricket matches at Gaddafi Stadium, and political jalsas — routinely draw tens of thousands of people into confined spaces. When density exceeds a safe threshold (commonly cited as 4–5 persons per square metre), crowd-crush incidents become a real risk. Automated crowd counting converts raw camera feeds into an actionable number and a spatial density field that operators can monitor in real time.

Early approaches to crowd counting relied on detecting individual heads or bodies with object detectors. These methods break down in dense crowds because of severe occlusion and scale variation. The field therefore moved toward density-map regression: instead of detecting people, a convolutional neural network (CNN) learns

to predict a continuous density surface; summing the surface over an image region yields the count. Multi-Column CNN (MCNN) and, more influentially, CSRNet demonstrated that a VGG-based front-end combined with dilated convolutions captures both fine head-scale detail and large receptive-field context, achieving strong accuracy on benchmark datasets. Our project adapts this well-established, citation-rich architecture into a practical, resource-light system with an operational dashboard layer.

Crucially, this proposal is scoped to be implementable by a four-member student team within a single semester. We deliberately freeze the heavy VGG-16 feature extractor and train only a small back-end, use down-sampled training crops, and rely on free GPU quotas (Google Colab / Kaggle). The novelty is therefore not a new state-of-the-art network but a well-engineered, end-to-end, deployable application that fuses an accepted research model with heatmaps, alerting and analytics.

4. Problem Statement

Public venues are monitored by CCTV operators who must visually estimate crowd size and judge when an area is becoming dangerously congested. This manual process suffers from four concrete problems:

- **Inaccuracy:** Human counting is inaccurate beyond a few dozen people and degrades further with occlusion, perspective distortion and poor lighting.
- **Scalability:** One operator cannot continuously watch many feeds; fatigue and attention lapses cause missed build-ups.
- **Latency:** By the time over-crowding is noticed manually, the safe window for crowd dispersal may already have closed.
- **No analytics:** There is rarely a quantitative log of crowd levels over time for after-action review or capacity planning.

Formally: given a CCTV image (or video stream) I , estimate (a) the total head count N , (b) a spatial density map D such that $\sum D \approx N$, and (c) a binary over-crowding flag when local density or total count exceeds configurable thresholds — all in near real time and on commodity hardware. Existing research models solve (a) and (b) but are rarely packaged as a usable monitoring tool; this project closes that gap.

5. Objectives

The project pursues the following measurable objectives:

- **Primary objective:** Design and implement a density-map regression model (CSRNet-style, transfer learning) that estimates crowd count from a single CCTV frame with $MAE \leq 95$ on ShanghaiTech Part A and ≤ 15 on the Mall dataset.
- **Secondary objective:** Generate and overlay a colour density heatmap that visually highlights the most congested regions of the scene.
- **Secondary objective:** Build a configurable over-crowding alert engine (per-region and global thresholds) that raises a visual/audible warning and logs the event.
- **Secondary objective:** Develop a real-time analytics dashboard (Streamlit/Flask) showing live count, density heatmap, time-series trend and alert history.
- **Secondary objective:** Achieve near real-time throughput (≥ 8 FPS on Colab GPU / mid-range laptop GPU) through input down-scaling and a lightweight back-end.

- **Learning objective:** Demonstrate competence in CNNs, transfer learning, density-map ground-truth generation, model evaluation (MAE/MSE) and ML application deployment.

6. Scope of the Project

In Scope

- Image- and video-file based crowd counting (offline and near real-time playback).
- Single-camera, roughly fixed-viewpoint indoor/outdoor scenes similar to the training datasets.
- Density heatmap visualisation and PNG/CSV export of results.
- Threshold-based over-crowding alerts with an event log.
- A web dashboard with live count, heatmap, time-series chart and alert history.
- Model training/fine-tuning on ShanghaiTech and Mall datasets using free cloud GPUs.

Out of Scope (Limitations)

- Multi-camera 3-D person re-identification and tracking of specific individuals.
- Face recognition or any biometric identification (excluded for privacy and ethical reasons).
- Guaranteed accuracy on extreme scenes far outside the training distribution (e.g. aerial drone shots).
- Production-grade deployment, 24/7 uptime, or integration with proprietary VMS/CCTV servers.
- Edge-device (Raspberry Pi) deployment — feasible as a future enhancement but not a deliverable.

7. Literature Review

Crowd counting has evolved through three broad paradigms — detection-based, regression-based (global count regression), and density-map estimation — with the latter now dominant. The following review summarises the most relevant works and benchmark resources.

7.1 Foundational and Recent Works

Zhang et al. (2016) introduced the Multi-Column CNN (MCNN), using three CNN columns with different filter sizes to handle head-scale variation, and released the ShanghaiTech dataset that is still a primary benchmark. Li et al. (2018) proposed CSRNet — a VGG-16 front-end followed by dilated convolutions in the back-end — which removed the redundant multi-column design, enlarged the receptive field without losing resolution, and substantially reduced MAE; CSRNet is the architectural basis for this project. Subsequent work pushed accuracy further: Liu et al. (2019) Context-Aware Network (CAN) adaptively fuses multi-scale contextual features; Ma et al. (2019) Bayesian Loss reframes supervision around point annotations rather than blurred density maps; and Wang et al. (2020) DM-Count uses optimal-transport-based loss for sharper density estimates. Transformer-based counters (e.g. TransCrowd, 2021) and crowd-localisation methods continue to appear, but they are heavier to train and unnecessary for our resource-constrained scope.

For the application layer, OpenCV provides classical image I/O and pre-processing, while lightweight deployment tools (Streamlit, Flask, Gradio) make it straightforward to wrap a trained model in a real-time dashboard. The YOLO family (v5-v8) can serve as an optional sparse-scene person detector for cross-checking counts in low-density frames.

7.2 Comparison of Crowd-Counting Approaches

Method (Year)	Core Idea	Strength	Limitation / Cost	Reported MAE (SH-A)
Head detection + counting	Detect each person/head	Interpretable; works when sparse	Fails under heavy occlusion; slow	≈ 90-110
MCNN (2016)	3-column CNN, multi-scale	Handles scale variation; small model	Lower accuracy; blurry maps	110.2
CSRNet (2018) ★ used	VGG-16 + dilated conv back-end	High accuracy; single stream; transfer-friendly	VGG front-end is large (frozen here)	68.2
CAN (2019)	Context-aware multi-scale fusion	Better on scale-varying scenes	More complex training	62.3
Bayesian Loss (2019)	Point-supervision loss	No need for tuned Gaussian maps	Loss implementation overhead	62.8
DM-Count (2020)	Optimal-transport loss	Sharper, well-calibrated maps	Heavier loss computation	59.7
TransCrowd (2021)	Vision-Transformer regression	Strong global context	Large model; needs more GPU/data	≈ 66

Table 1.1: Representative crowd-counting methods. ★ = approach adopted (in a frozen-front-end, low-resource configuration). MAE values are as reported by the original papers on ShanghaiTech Part A.

7.3 Benchmark Datasets

Dataset	Scale / Content	Annotation	Why Useful Here
ShanghaiTech Part A	482 images, very dense web crowds	Head-point annotations	Primary benchmark; stress-tests dense counting
ShanghaiTech Part B	716 images, street scenes (sparser)	Head-point annotations	Closer to CCTV viewpoints; faster to train
Mall dataset	2 000 frames, single fixed camera	Head-point annotations	Realistic CCTV-style; small enough for laptops
UCF-QNRF / JHU-CROWD++	Large, extremely dense (optional)	Head-point annotations	Optional extra robustness; large download

Table 1.2: Public crowd-counting datasets considered for training and evaluation.

Research gap addressed: while accuracy-oriented papers abound, comparatively little student-level work packages a proven counter into a usable, real-time monitoring dashboard with alerting and historical analytics on free hardware. This project targets exactly that engineering gap.

8. Proposed Methodology

The system follows a classic train-once / serve-many design. The methodology is divided into a ground-truth preparation stage, a model training stage, and an application/serving stage.

8.1 System Architecture

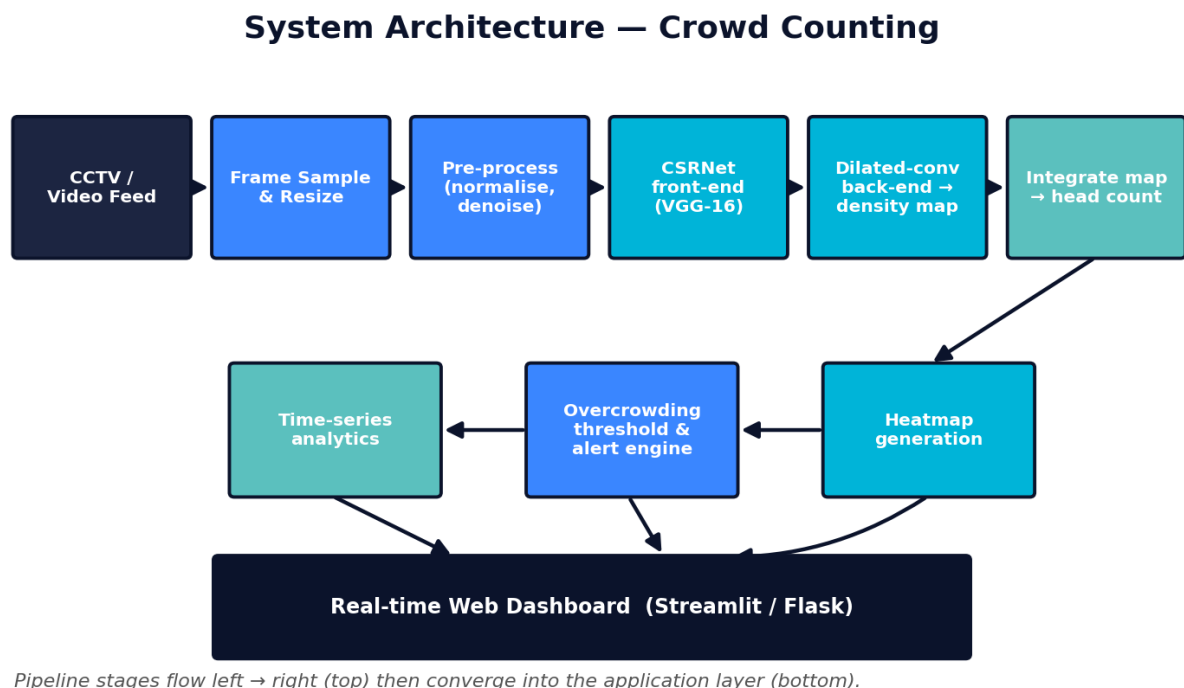


Figure 1.1: End-to-end system architecture — from CCTV feed through the CSRNet density model to heatmap, alerting and the analytics dashboard.

8.2 Step-by-Step Workflow

1. **Data acquisition** — Download ShanghaiTech (Part A & B) and the Mall dataset; organise images and head-point annotation files (.mat / .json).
2. **Ground-truth density maps** — Convert each head-point annotation into a density map by placing a normalised 2-D Gaussian kernel at every head location (geometry-adaptive σ for dense scenes, fixed σ for sparser ones). The integral of the map equals the true count.
3. **Pre-processing & augmentation** — Resize/crop images (e.g. 384–512 px on the short side), normalise with ImageNet statistics, and augment with random crops, horizontal flips and mild brightness/contrast jitter; rescale density maps consistently.
4. **Model construction** — Build CSRNet: load VGG-16 (first 10 conv layers, ImageNet-pretrained) as a frozen front-end; attach a back-end of six dilated 3×3 convolutions (dilation 2) and a 1×1 conv producing a single-channel density map at 1/8 resolution.

5. **Training** — Optimise pixel-wise Euclidean (MSE) loss between predicted and ground-truth density maps using Adam/SGD with a small learning rate; train on a free Colab/Kaggle GPU with checkpointing; monitor validation MAE and MSE.
6. **Evaluation** — Report MAE and RMSE on the held-out test split; visualise predicted vs. ground-truth density maps; compare against an MCNN baseline.
7. **Count extraction & heatmap** — Sum the predicted density map for the count; upsample and colour-map (jet/viridis) it, then alpha-blend over the original frame for the heatmap overlay.
8. **Alert engine** — Compare global count and per-cell density against user-set thresholds; if exceeded, raise a dashboard alert (colour change + sound), and append {timestamp, count, location} to a log.
9. **Dashboard & integration** — Wrap everything in a Streamlit app: upload image/video, see live count, heatmap, a rolling time-series chart and an alert table; allow CSV/PNG export.
10. **Testing & optimisation** — Measure FPS, tune input resolution, optionally add YOLO cross-check for sparse frames, and document results.

8.3 Workflow Flowchart

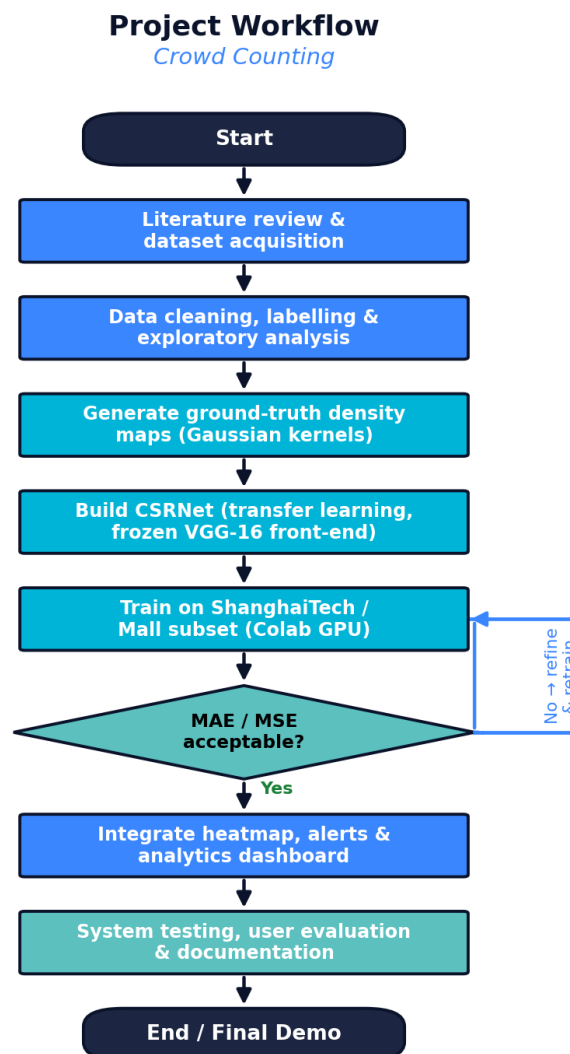


Figure 1.2: Project workflow with the model-quality decision gate and retraining loop.

8.4 Algorithms, Models & Frameworks

- **CSRNet (primary):** VGG-16 front-end (frozen) + dilated-convolution back-end → single-channel density map.
- **MCNN (baseline):** Lightweight multi-column CNN used as a comparison baseline.
- **YOLOv8-n (optional):** Tiny person detector to cross-validate counts in low-density frames.
- **Loss & metrics:** Pixel-wise MSE training loss; evaluation by MAE and RMSE on counts.
- **Frameworks:** PyTorch (or TensorFlow/Keras), OpenCV, NumPy, SciPy (Gaussian kernels), Matplotlib, Streamlit/Flask.

9. Technologies and Tools

Category	Tools / Technologies	Purpose in the Project
Programming language	Python 3.10+	Primary language for data prep, modelling and the dashboard
Deep-learning framework	PyTorch (alt. TensorFlow / Keras)	Building, training and serving CSRNet/MCNN
Computer vision	OpenCV, Pillow	Frame extraction, resizing, colour-mapping, video I/O
Numerical / scientific	NumPy, SciPy	Array ops; geometry-adaptive Gaussian density-map generation
Pretrained models	torchvision VGG-16 (ImageNet), Ultralytics YOLOv8-n	Transfer-learning front-end; optional sparse-scene detector
Visualisation	Matplotlib, Plotly	Density-map plots, time-series charts in the dashboard
App / dashboard	Streamlit (alt. Flask + Bootstrap)	Real-time upload, count, heatmap, alerts, analytics UI
Training environment	Google Colab / Kaggle Notebooks (free GPU)	GPU training without local high-end hardware
Dev tools	VS Code, Jupyter, Git/GitHub	Coding, experimentation, version control, collaboration
Dataset sources	ShanghaiTech, Mall dataset, (opt.) UCF-QNRF	Training, validation and benchmarking data

Table: Technology stack selected for cost-free, low-resource development.

10. Dataset Information

10.1 Candidate Datasets

- **ShanghaiTech Part A:** $\approx$ 482 highly congested images crawled from the web; head-point annotations; the standard hard benchmark.

- **ShanghaiTech Part B:** ≈ 716 images of Shanghai street scenes; sparser, more CCTV-like; lighter to train.
- **Mall dataset:** $\approx 2\,000$ frames from a single fixed shopping-mall camera; ideal for a realistic, lightweight prototype.
- **Optional: UCF-QNRF / JHU-CROWD++:** Very large, extremely dense datasets for an additional robustness test if time and bandwidth permit.

10.2 Data Pre-processing

- Parse head-point annotations and build per-image density maps using normalised 2-D Gaussian kernels (geometry-adaptive σ for dense Part A; fixed $\sigma \approx 4$ px for sparser Part B / Mall).
- Resize images so the short side is ≈ 384 – 512 px (keeping aspect ratio); scale density maps to the network output resolution ($1/8$) and rescale values so the integral is preserved.
- Normalise pixels with ImageNet mean/standard deviation to match the pretrained VGG-16 front-end.
- Clean corrupt/duplicate files; verify that $\sum(\text{density map}) \approx \text{annotated count}$ for every sample.

10.3 Training / Validation / Testing Split

Split	Proportion	Purpose
Training	$\approx 70\%$	Learn back-end weights; the official train sets are used where provided
Validation	≈ 10 – 15%	Hyper-parameter tuning, early stopping, model selection
Testing	≈ 15 – 20%	Final unbiased MAE/RMSE; official test split used when available

Table 1.3: Data partitioning. Official ShanghaiTech splits are respected; the Mall dataset is split chronologically to avoid leakage between adjacent frames.

10.4 Data Augmentation

Because dense crowd datasets are relatively small, augmentation is essential: random crops (e.g. 256×256 / quarter-image patches), random horizontal flips, mild brightness/contrast/colour jitter, and occasional Gaussian noise or blur to mimic low-quality CCTV. All geometric transforms are applied identically to the image and its density map. Synthetic crowd imagery (e.g. GCC-style renders) can optionally be added for pre-training.

11. Expected Outcomes

- A trained CSRNet-style density model with MAE ≈ 70 – 95 on ShanghaiTech Part A and ≤ 15 on the Mall dataset (competitive with classic baselines, well within student reach).
- Accurate per-frame head counts plus a colour density heatmap overlay highlighting congestion hotspots.

- A working over-crowding alert engine with configurable thresholds and a timestamped event log.
- A real-time Streamlit dashboard: live count, heatmap, rolling time-series trend, alert history, CSV/PNG export.
- Near real-time throughput ($\approx 8\text{--}15$ FPS on Colab/mid-range GPU).
- Complete documentation: code repository, training notebooks, evaluation report and user guide.

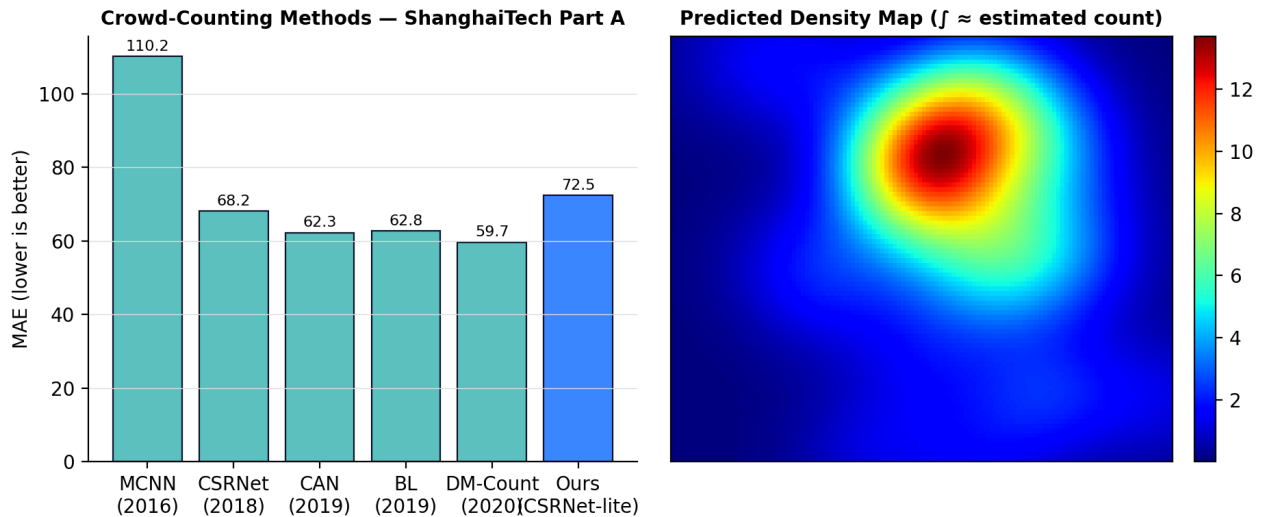


Figure 1.3: (left) MAE of representative methods on ShanghaiTech Part A with our expected operating point; (right) an example predicted density map whose integral approximates the crowd count.

12. Innovation / Uniqueness

- **Research-to-product packaging:** Turns a citation-rich research model (CSRNet) into a complete, operator-facing monitoring tool — count + heatmap + alerts + analytics — which most student projects omit.
- **Low-resource engineering:** Frozen VGG-16 front-end, down-scaled crops and free cloud GPUs make state-of-research ideas reproducible on ordinary laptops.
- **Configurable safety layer:** Per-region and global density thresholds let the same model serve very different venues (mall corridor vs. stadium gate).
- **Privacy-by-design:** Counts density, never identities — no face recognition — addressing a real ethical concern in public surveillance.
- **Analytics & after-action review:** Time-series logging supports capacity planning and incident review, not just live monitoring.
- **Optional hybrid cross-check:** A tiny YOLOv8-n detector validates counts in sparse frames, improving trust in the output.

13. Feasibility Analysis

13.1 Technical Feasibility

CSRNet, MCNN and YOLO have open-source reference implementations; the datasets are public; and PyTorch/OpenCV/Streamlit are mature. Freezing the VGG-16 front-end reduces the trainable parameters to a few million, so a single free Colab GPU session (T4, ~ 16 GB) can fine-tune the back-end in a few hours. Inference on down-scaled frames runs comfortably in near real time.

13.2 Cost Analysis

Item	Option Chosen	Cost (PKR)
Compute for training	Google Colab / Kaggle free GPU tier	0
Datasets	ShanghaiTech, Mall (public, free)	0
Software & libraries	Python, PyTorch, OpenCV, Streamlit (open-source)	0
Development machines	Members' existing laptops	0 (already owned)
Optional Colab Pro (faster GPU)	Only if needed for final tuning	≈ 2 800 / month (optional)
Hosting the demo (optional)	Streamlit Community Cloud / Hugging Face Spaces (free)	0
Estimated total	Core project	≈ 0 (PKR 0–3 000 if Colab Pro used)

Table 1.4: Indicative cost analysis — the project is essentially zero-budget.

13.3 Hardware Feasibility & GPU / Resource Limitations

- Training: a free Colab/Kaggle GPU (NVIDIA T4/P100) is sufficient; full UCF-QNRF training is avoided due to size — Mall and ShanghaiTech are the working sets.
- Local development/inference: any laptop with 8 GB+ RAM; a 4 GB+ GPU is helpful but the model also runs on CPU at reduced FPS for demos.
- Mitigations for limited resources: smaller input resolution, gradient accumulation, mixed-precision training, checkpoint-and-resume to survive Colab session limits, and caching pre-computed density maps.

13.4 Practical Implementation Plan

Weeks 1–4: literature, dataset download, density-map pipeline, MCNN baseline. Weeks 5–8: build and train CSRNet on Colab, iterate to target MAE. Weeks 9–12: heatmap module, alert engine, Streamlit dashboard. Weeks 13–16: integration, FPS optimisation, testing, evaluation report, documentation and final demo. Work is split as: data/ground-truth (member A), model training (members A & B), dashboard/heatmap (member C), alerts/analytics & documentation (member D), with weekly integration checkpoints.

14. Hardware and Software Requirements

Requirement	Minimum	Recommended
Processor	Quad-core CPU (Intel i5 / Ryzen 5)	8-core CPU
RAM	8 GB	16 GB
GPU (local, optional)	Integrated / 2 GB GPU (CPU inference OK)	NVIDIA GPU 4–6 GB+ (e.g. GTX 1650)

GPU (training)	Google Colab / Kaggle free T4	Colab Pro / institutional GPU
Storage	20 GB free	50 GB free (datasets + checkpoints)
OS	Windows 10 / Ubuntu 20.04 / macOS	Ubuntu 22.04
Software	Python 3.10, PyTorch, OpenCV, Streamlit, Git	+ CUDA toolkit (if local GPU), VS Code
Connectivity	Broadband for dataset/model downloads	Stable connection for Colab sessions

Table 1.5: Hardware and software requirements.

15. Future Enhancements

- Multi-camera fusion and approximate scene-level (homography-based) density maps.
- Edge deployment: model pruning/quantisation (TensorRT, ONNX, TFLite) for Jetson Nano / Raspberry Pi.
- Temporal smoothing with a lightweight RNN/Kalman filter for steadier video counts.
- Predictive crowding: short-horizon forecasting of density trends to warn before thresholds are hit.
- Anomaly detection (sudden surges, counter-flows) layered on the density stream.
- Mobile/operator app with push notifications; integration with standard VMS via RTSP.
- Domain adaptation / few-shot fine-tuning for a specific venue with minimal new labels.

16. Timeline / Gantt Chart

The project spans a $\approx$ 16-week semester, organised into a foundation phase, a modelling & development phase, and a finalisation phase. Weekly milestones are shown below.

Project Timeline / Gantt Chart — Crowd Counting

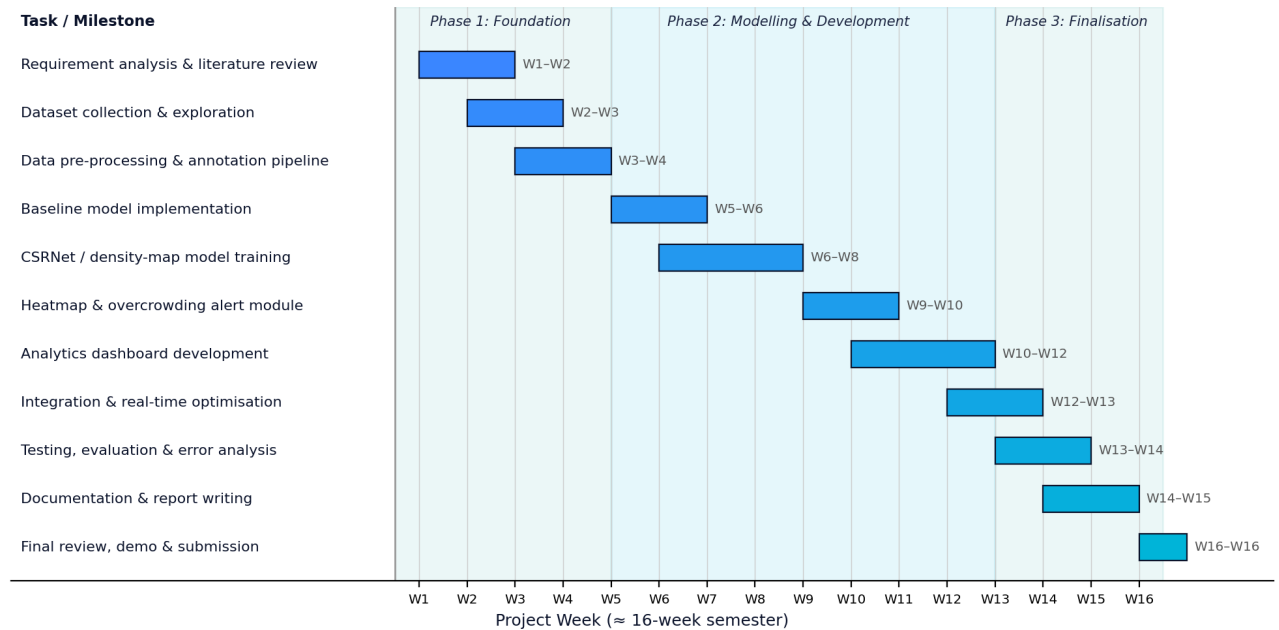


Figure 1.4: Project Gantt chart with weekly milestone breakdown.

Week(s)	Milestone / Deliverable
W1-W2	Requirement analysis, literature review, finalise architecture and tools
W2-W3	Download & organise ShanghaiTech and Mall datasets; exploratory analysis
W3-W4	Build density-map ground-truth pipeline; data-loader and augmentation
W5-W6	Implement MCNN baseline; set up Colab training environment
W6-W8	Build and train CSRNet (frozen VGG-16 + dilated back-end); reach target MAE
W9-W10	Heatmap generation module + over-crowding alert engine with event logging
W10-W12	Streamlit analytics dashboard (live count, heatmap, time-series, alerts)
W12-W13	End-to-end integration; near-real-time optimisation (resolution, batching)
W13-W14	Testing, evaluation, error analysis, comparison tables and charts
W14-W15	Documentation, user guide and final report writing
W16	Final review, demonstration and submission

Table 1.6: Weekly milestone breakdown.

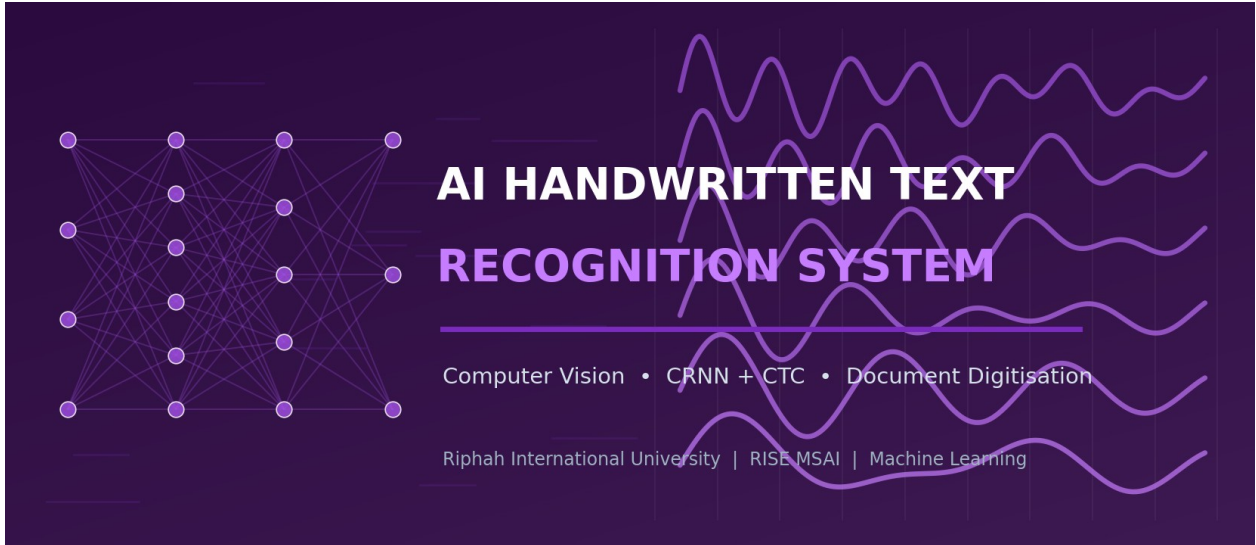
17. Conclusion

This proposal presented an AI-Based Crowd Counting System that converts ordinary CCTV footage into real-time head counts, a density heatmap, automated over-crowding alerts and historical analytics. The technical core — a CSRNet-style density-map regressor with a frozen, pretrained VGG-16 front-end — is well established in the literature yet light enough to train on free cloud GPUs and run on commodity laptops. The project's contribution is engineering and integration: wrapping a proven model in a privacy-respecting, operator-friendly dashboard. It is realistic for a four-member team to deliver within one semester, develops core machine-learning competencies, and produces a genuinely useful public-safety tool with a clear path to future enhancement.

18. References

References are listed in IEEE citation style; arXiv / dataset URLs are provided where applicable.

- [1] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 589–597.
- [2] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in Proc. IEEE/CVF CVPR, 2018, pp. 1091–1100. [Online]. Available: <https://arxiv.org/abs/1802.10062>
- [3] W. Liu, M. Salzmann, and P. Fua, "Context-aware crowd counting," in Proc. IEEE/CVF CVPR, 2019, pp. 5099–5108.
- [4] Z. Ma, X. Wei, X. Hong, and Y. Gong, "Bayesian loss for crowd count estimation with point supervision," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2019, pp. 6142–6151.
- [5] B. Wang, H. Liu, D. Samaras, and M. Hoai, "Distribution matching for crowd counting (DM-Count)," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1595–1607.
- [6] D. Liang, X. Chen, W. Xu, Y. Zhou, and X. Bai, "TransCrowd: Weakly-supervised crowd counting with transformers," Science China Information Sciences, vol. 65, no. 6, 2022.
- [7] V. A. Sindagi and V. M. Patel, "A survey of recent advances in CNN-based single image crowd counting and density estimation," Pattern Recognition Letters, vol. 107, pp. 3–16, 2018.
- [8] C. Wang, H. Zhang, L. Yang, S. Liu, and X. Cao, "Deep people counting in extremely dense crowds," in Proc. ACM Int. Conf. Multimedia (ACM MM), 2015, pp. 1299–1302. (ShanghaiTech / dense-crowd context.)
- [9] G. Jocher et al., "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [10] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in NeurIPS, 2019, pp. 8024–8035.
- [11] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [12] Papers With Code, "ShanghaiTech crowd-counting dataset." [Online]. Available: <https://paperswithcode.com/dataset/shanghaitech>
- [13] C. C. Loy, S. Gong, and T. Xiang, "From semi-supervised to transfer counting of crowds (Mall dataset)," in Proc. IEEE ICCV, 2013, pp. 2256–2263.
- [14] Streamlit Inc., "Streamlit — A faster way to build and share data apps," documentation, 2024. [Online]. Available: <https://docs.streamlit.io>



FINAL PROJECT PROPOSAL

AI-Based Handwritten Text Recognition System

Proposal 2 of 3 — A Final-Year Machine-Learning Project Proposal

Riphah International University

Department of RISE MSAI

Course / Subject: Machine Learning

Submitted By — Group Members	Submitted To
1. Azhar Hussain <i>(Group Lead)</i> 2. Rahma 3. Haris Javaid 4. Syed Farakh Abbas	Junaid Khan <i>Course Instructor — Machine Learning</i>

Date of Submission: 11 May 2026

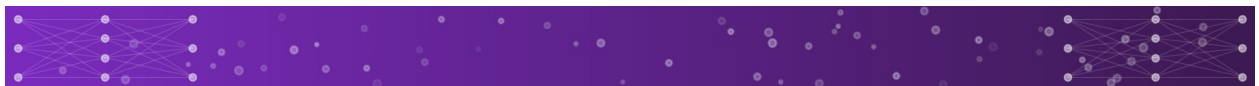
Academic Session 2025–2026 | Confidential — For Academic Evaluation Only

2.0. Contents — Proposal 2: AI-Based Handwritten Text Recognition System

This proposal contains the standard sections (Abstract through References) as listed in the Master Table of Contents at the front of this document.

2. Abstract

Vast amounts of human knowledge — lecture notes, examination scripts, historical manuscripts, medical records, official registers, filled forms — still exist only as handwriting on paper. Converting this content into editable, searchable digital text unlocks indexing, translation, accessibility and long-term preservation. This project proposes an AI-Based Handwritten Text Recognition (HTR) system that takes an image, scanned page or PDF of handwritten text and outputs editable text, exportable as TXT, DOCX or a searchable PDF. The recognition engine is a Convolutional Recurrent Neural Network (CRNN): a CNN feature extractor (a compact 7-layer stack or a MobileNet-style backbone) feeds a bidirectional LSTM sequence encoder, which is decoded with Connectionist Temporal Classification (CTC) — the architecture introduced by Shi et al. for image-based sequence recognition. Around this core the system adds image pre-processing (deskewing, binarisation, line and word segmentation), a dictionary/edit-distance spell-correction post-processor (SymSpell), basic layout reconstruction, and an optional second model for Urdu/regional scripts. The system is trained on the IAM Handwriting Database (and synthetic data generated from handwriting fonts) using a free Google Colab GPU. Targeted performance is a word-level Character Error Rate (CER) of roughly 8–12 % on IAM lines without a language model and lower with post-processing — comfortably achievable by a student team — together with a usable desktop/web application for everyday note digitisation.



Keywords: *Handwritten text recognition, OCR, CRNN, CTC loss, BiLSTM, transfer learning, document digitisation, IAM database, multi-language OCR.*

3. Introduction

Optical Character Recognition (OCR) for printed text is largely a solved problem, but handwritten text recognition remains challenging because handwriting is highly variable — slant, spacing, letter shapes, ligatures and pen pressure differ between writers and even within one writer's page. Classical HTR pipelines first segmented a line into individual characters and classified each one, a brittle approach that fails when letters touch or overlap. The breakthrough came with segmentation-free sequence learning: an image of an entire word or text line is processed by a CNN to extract a feature sequence, a recurrent network models the left-to-right context, and a CTC layer aligns the variable-length prediction to the variable-length label without needing per-character segmentation. Shi, Bai and Yao's CRNN (2015) established this end-to-end design and remains the practical workhorse of HTR; later work (Puigcerver 2017; Bluche & Messina 2017) refined the convolutional and recurrent blocks, and

recent Transformer models such as TrOCR (2021) push accuracy further at much higher computational cost.

This project deliberately builds on the CRNN+CTC design because it is accurate, well documented, and trainable on a free cloud GPU. To make the system genuinely useful rather than a bare model, we add a complete document pipeline: pre-processing (deskew, binarise, denoise), line/word segmentation, the CRNN recogniser, a spell-correction post-processor, simple layout reconstruction, and export to editable formats. Multi-language support (an English model plus an optional Urdu/Arabic-script model) and a clean user interface complete the deliverable.

4. Problem Statement

Organisations and individuals accumulate large volumes of handwritten material that cannot be searched, edited, translated or efficiently archived. Manual transcription is the current fallback, but it is:

- **Costly:** Extremely time-consuming and expensive at scale (thousands of pages of notes, forms, registers).
- **Inaccurate:** Error-prone and inconsistent, especially for unfamiliar handwriting or domain jargon.
- **Inaccessible:** A barrier to accessibility — handwritten notes are not screen-reader friendly for visually impaired users.
- **Fragile:** An obstacle to preservation — paper degrades, and without digitisation the content is at risk.

Formally: given an input image / scan / PDF page P containing handwritten text, produce an editable digital transcript T that closely matches the written content (low character and word error rates), preserving basic layout (lines, paragraphs), supporting more than one language/script, and exporting to TXT / DOCX / searchable PDF — all runnable on student hardware. Research models address the core recognition step; this project additionally delivers the surrounding, user-facing document workflow.

5. Objectives

- **Primary objective:** Implement a CRNN + CTC handwriting recogniser achieving a line-level CER of roughly 8–12 % on the IAM test set without a language model (lower with post-processing).
- **Secondary objective:** Build a robust pre-processing stage: orientation correction, binarisation, denoising, and line/word segmentation for multi-line documents.
- **Secondary objective:** Add a dictionary + edit-distance spell-correction post-processor (SymSpell) to reduce word-level errors.
- **Secondary objective:** Provide multi-language support — an English model plus an optional Urdu/Arabic-script model — with a simple language selector.
- **Secondary objective:** Reconstruct basic layout (line order, paragraphs) and export results as TXT, DOCX and searchable PDF.
- **Secondary objective:** Deliver a clean desktop/web UI for uploading images/PDFs and reviewing/editing the recognised text, including a near-real-time webcam capture mode.

- **Learning objective:** Demonstrate competence in CNNs, RNN/LSTM sequence modelling, CTC loss, transfer learning, data augmentation and OCR evaluation (CER/WER).

6. Scope of the Project

In Scope

- Recognition of handwritten Latin-script (English) text from images, scans and PDF pages.
- Pre-processing: deskew, binarise, denoise, and line/word segmentation for multi-line documents.
- CRNN+CTC recognition with greedy and beam-search decoding; SymSpell post-correction.
- Optional second model for Urdu/Arabic-script words (proof-of-concept, smaller dataset).
- Basic layout reconstruction and export to TXT, DOCX and searchable PDF.
- A desktop/web application with file upload, webcam capture and an editable results view.

Out of Scope (Limitations)

- Full historical-manuscript transcription with complex multi-column layouts, marginalia and tables.
- Cursive recognition in arbitrary languages beyond the trained models; mathematical-formula recognition.
- Guaranteed accuracy on extremely messy, faded or low-resolution handwriting outside the training distribution.
- Large-scale Transformer models (e.g. full TrOCR fine-tuning) — noted as a future enhancement only.
- Production-grade hosting, user accounts and document-management features.

7. Literature Review

Handwritten text recognition has progressed from per-character segmentation, through HMM- and MDLSTM-based line recognisers, to the now-standard CNN+RNN+CTC family and, most recently, Transformer-based encoder-decoder models. The works below frame our design choices.

7.1 Key Works

Graves et al. (2009) showed that multi-dimensional LSTMs with a CTC output layer could recognise unconstrained handwriting at the line level without explicit segmentation, winning ICDAR competitions. Shi, Bai and Yao (2015) introduced the CRNN — convolutional layers for feature extraction, bidirectional LSTMs for sequence modelling, and a CTC transcription layer — as a unified, end-to-end trainable network for image-based sequence recognition; this is the backbone of our system. Puigcerver (2017) demonstrated that carefully designed 1-D (rather than 2-D) recurrent layers on convolutional features are sufficient and faster, setting strong IAM baselines, while Bluche and Messina (2017) proposed gated CNN-BLSTM architectures for multilingual HTR. Graves (2012) and Graves et al. (2006) provide the theoretical basis of CTC. More recently, TrOCR (Li et al., 2021) couples a pretrained vision Transformer encoder

with a pretrained text Transformer decoder, achieving very low error rates but requiring substantial compute and large pretraining corpora — beyond our scope, though an obvious future direction. Open-source engines such as Tesseract 4/5 (LSTM-based) and toolkits like PyLaia, Kraken and PaddleOCR offer reusable components and baselines. SymSpell (Garbe) supplies fast symmetric-delete spelling correction for the post-processing stage.

7.2 Comparison of HTR Approaches

Approach (Year)	Core Idea	Strength	Limitation / Cost	IAM CER (approx.)
Char-segmentation + CNN	Segment line→chars, classify each	Simple components	Brittle; touching letters break it	high / unstable
MDLSTM + CTC (2009)	2-D LSTM over the image + CTC	Segmentation-free; historic SOTA	Slow; hard to train	≈ 18 %
CRNN (CNN+BLSTM+CTC) (2015) ★	CNN features → BiLSTM → CTC	Accurate, end-to-end, light, well-documented	Needs decent data & augmentation	≈ 10–12 %
Puigcerver CNN-1D-LSTM (2017)	Conv features + 1-D LSTMs	Faster; strong IAM baseline	Still needs careful tuning	≈ 8 %
Gated CNN-BLSTM (2017)	Gated conv + BLSTM, multilingual	Good cross-language results	More complex architecture	≈ 8 %
TrOCR (2021)	ViT encoder + Transformer decoder	Very low error; no CTC needed	Large model; heavy pretraining/compute	≈ 3–4 %
Tesseract 4/5 (LSTM)	Open-source LSTM line recogniser	Free, ready-to-use, many languages	Weaker on free-form handwriting	varies

Table 2.1: Representative HTR approaches. ★ = backbone adopted (CRNN + CTC, with a lightweight CNN front-end). CER figures are indicative line-level results reported in the literature on the IAM database.

7.3 Datasets & Resources

Resource	Content	Use Here
IAM Handwriting Database	≈ 1 539 scanned forms, 13 353 labelled lines, 657 writers (English)	Primary training/validation/testing data
IAM-OnDB / CVL / RIMES	Additional handwriting corpora (online strokes / multi-writer / French)	Optional extra data and robustness checks
Synthetic handwriting (TRDG / fonts)	Text rendered with many handwriting fonts + distortions	Pre-training and augmentation to enlarge the training set
UrduDoc / UNHD / KHATT (Arabic-script)	Urdu/Arabic handwritten text datasets	Optional second-language proof-of-concept
SymSpell dictionary, NLTK/WordFreq	Word lists and frequencies	Spell-correction post-processing

Table 2.2: Datasets and resources considered.

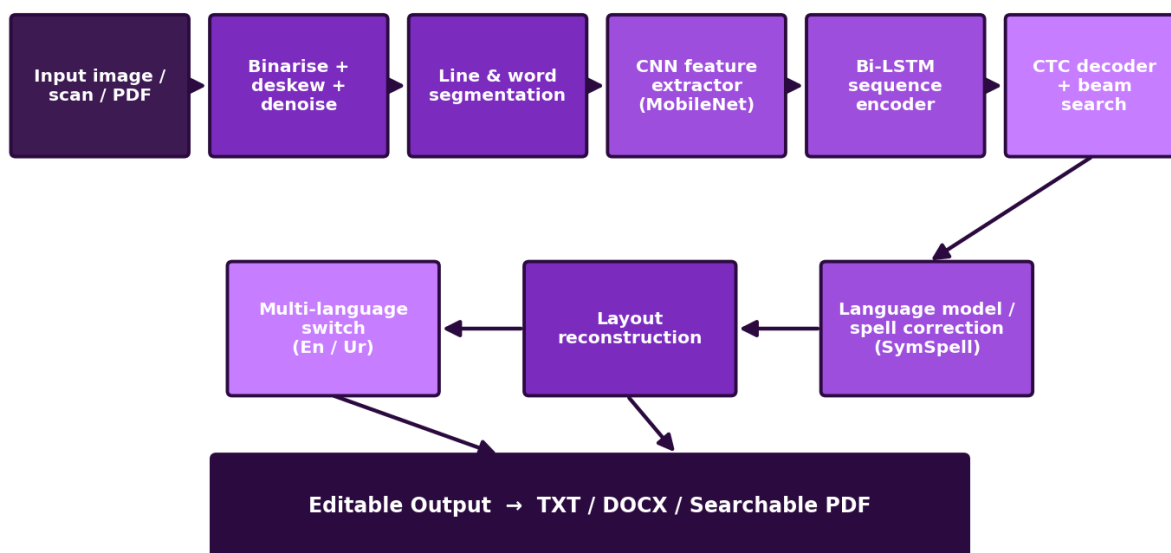
Research gap addressed: most academic HTR work optimises the recogniser in isolation on IAM lines. Far fewer student projects deliver the full document workflow — pre-processing, segmentation, post-correction, layout rebuild, multi-language, and export — wrapped in a usable app. This project targets that practical, end-to-end gap on free hardware.

8. Proposed Methodology

The methodology has three stages: a document pre-processing/segmentation stage, a CRNN+CTC recognition stage (trained once, served many times), and a post-processing/export/UI stage.

8.1 System Architecture

System Architecture — Handwritten Text Recognition



Pipeline stages flow left → right (top) then converge into the application layer (bottom).

Figure 2.1: End-to-end architecture — image/scan/PDF → pre-processing & segmentation → CRNN (CNN + BiLSTM + CTC) → spell-correction & layout rebuild → editable TXT/DOCX/searchable-PDF output.

8.2 Step-by-Step Workflow

11. **Input handling** — Accept JPG/PNG images, scanned pages, multi-page PDFs (rasterised), or near-real-time webcam frames.
12. **Pre-processing** — Convert to grayscale; correct skew (Hough/projection-profile); binarise (Otsu / adaptive / Sauvola); remove noise and ruled lines; normalise contrast.
13. **Segmentation** — Detect text regions, then segment into lines (horizontal projection / connected components) and, where needed, into words; normalise each line image height (e.g. 32–64 px).
14. **Recogniser construction** — Build the CRNN: a CNN feature extractor (compact 7-conv stack or MobileNet-style backbone, optionally ImageNet-initialised) → map-

to-sequence → 2 stacked bidirectional LSTM layers → linear layer over the character set → CTC.

15. **Training** — Train end-to-end with CTC loss on IAM (plus synthetic data) using Adam, learning-rate scheduling and gradient clipping on a free Colab/Kaggle GPU; checkpoint regularly; monitor validation CER/WER.
16. **Decoding** — Use greedy (best-path) decoding for speed and CTC beam search (optionally with a small n-gram language model) for accuracy.
17. **Post-processing** — Apply SymSpell dictionary correction; restore casing/punctuation heuristics; reassemble words → lines → paragraphs in reading order.
18. **Multi-language** — Provide a language selector that swaps in an alternative model (e.g. an Urdu/Arabic-script CRNN) and an appropriate dictionary.
19. **Export & UI** — Render the result in an editable text panel beside the source image; export to TXT, DOCX (python-docx) and searchable PDF (OCR text layer over the image).
20. **Testing & evaluation** — Measure CER/WER on held-out IAM data and on a small custom test set of team members' handwriting; analyse error types; document results.

8.3 Workflow Flowchart

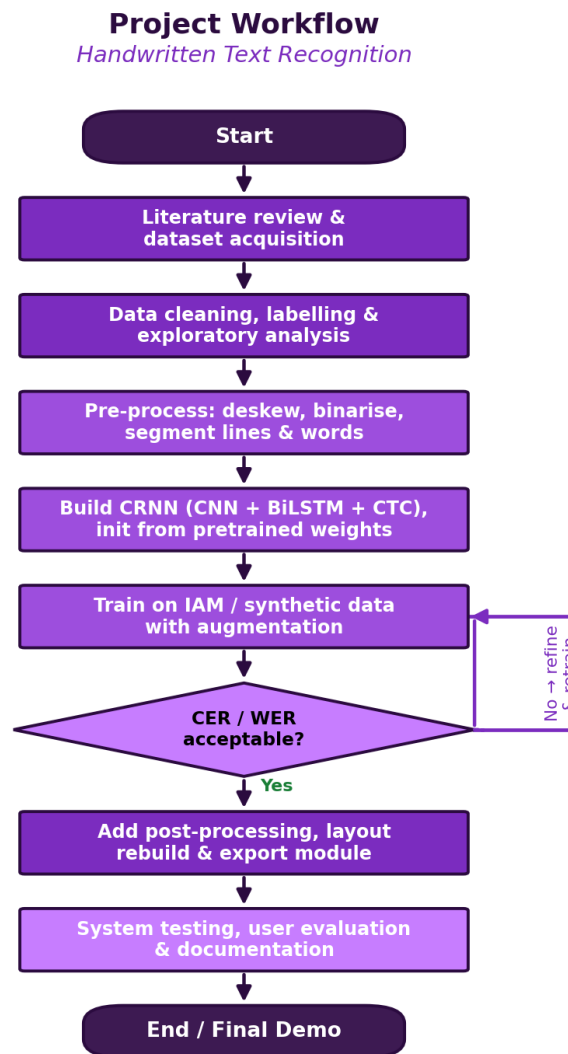


Figure 2.2: Project workflow with the model-quality decision gate and refinement loop.

8.4 Algorithms, Models & Frameworks

- **CRNN (primary):** CNN feature extractor + 2× BiLSTM + CTC transcription layer.
- **CTC loss / decoders:** Connectionist Temporal Classification training loss; greedy and beam-search decoding.
- **CNN backbone options:** Compact 7-layer conv stack (VGG-like) or MobileNetV3-small features (transfer learning).
- **Pre-processing algorithms:** Otsu/Sauvola binarisation, projection-profile deskew & line segmentation, connected components.
- **Post-processing:** SymSpell symmetric-delete spelling correction; optional KenLM n-gram language model in beam search.
- **Baselines:** Tesseract 5 (LSTM) and a per-character CNN classifier for comparison.
- **Frameworks:** PyTorch / TensorFlow-Keras, OpenCV, NumPy, scikit-image, python-docx, pdf2image/PyMuPDF, Streamlit/PyQt.

9. Technologies and Tools

Category	Tools / Technologies	Purpose in the Project
Programming language	Python 3.10+	Data prep, modelling, post-processing, application
Deep-learning framework	PyTorch (alt. TensorFlow / Keras)	Building and training the CRNN; CTC loss
Computer vision / imaging	OpenCV, scikit-image, Pillow	Binarisation, skew, denoise, line/word segmentation
Numerical	NumPy	Array operations, feature handling
Pretrained backbones	torchvision MobileNetV3 / ResNet18 (ImageNet)	Optional transfer-learning CNN front-end
OCR baselines / tools	Tesseract 5, (ref.) PaddleOCR / PyLaia	Comparison baselines and reusable components
Post-processing / NLP	SymSpell, NLTK / wordfreq, (opt.) KenLM	Spell correction and language modelling
Document I/O & export	pdf2image / PyMuPDF, python-docx, reportlab	Read PDFs; export TXT/DOCX/searchable PDF
App / UI	Streamlit (alt. PyQt / Flask)	Upload, webcam capture, editable results view, export buttons
Training environment	Google Colab / Kaggle Notebooks (free GPU)	GPU training without local high-end hardware
Dev tools	VS Code, Jupyter, Git/GitHub	Coding, experiments, version control, collaboration
Dataset sources	IAM database; synthetic (TRDG); (opt.) Urdu/Arabic sets	Training, validation, testing, multi-language PoC

Table: Technology stack selected for cost-free, low-resource development.

10. Dataset Information

10.1 Candidate Datasets

- **IAM Handwriting Database:** $\approx 1\,539$ scanned forms $\rightarrow 13\,353$ labelled text lines from 657 writers (English, modern Latin script); the standard HTR benchmark.
- **Synthetic handwriting data:** Text strings rendered with dozens of handwriting fonts and random distortions (rotation, elastic warp, noise) using tools like TextRecognitionDataGenerator; used for pre-training and to enlarge the training set.
- **Optional extra corpora:** IAM-OnDB, CVL, RIMES for additional robustness; KHATT/UNHD/UrduDoc for an optional Urdu/Arabic-script proof-of-concept.
- **Custom test set:** A small set of scanned notes/forms in the team members' own handwriting for a realistic end-to-end evaluation.

10.2 Data Pre-processing

- Extract line images and ground-truth transcripts from the IAM XML; filter unreadable or wrongly-segmented samples; build a fixed character vocabulary (letters, digits, punctuation, space, CTC blank).
- Normalise each line image: grayscale, deskew, binarise (or keep grayscale), pad/resize to a fixed height (e.g. 32–64 px) while preserving aspect ratio; right-pad widths within a batch.
- Tokenise transcripts to integer label sequences; record sequence lengths for CTC.
- For multi-line documents at inference time: deskew the page, segment into lines via projection profiles/connected components, then feed each line to the model.

10.3 Training / Validation / Testing Split

Split	Proportion	Purpose
Training	$\approx 70\text{--}75\%$	Learn CRNN weights (IAM official train set + synthetic data)
Validation	$\approx 10\text{--}15\%$	Hyper-parameter tuning, early stopping, decoder/LM selection
Testing	$\approx 15\%$	Final unbiased CER/WER (IAM official test set + custom handwriting set)

Table 2.3: Data partitioning. The IAM writer-independent split is respected so that no writer appears in both training and testing.

10.4 Data Augmentation

Augmentation is critical because handwriting datasets are limited and writer-dependent: random rotations ($\pm 3\text{--}5^\circ$), slant/shear, elastic and grid distortions, random scaling and translation, morphological erosion/dilation (stroke thinning/thickening), brightness/contrast jitter, Gaussian/salt-pepper noise, and random background textures. Large volumes of synthetic handwriting can be generated to pre-train the network before fine-tuning on real IAM data.

11. Expected Outcomes

- A trained CRNN+CTC recogniser with line-level CER $\approx 8\text{--}12\%$ on IAM without a language model, improving with beam search + SymSpell post-correction.
- A working document pipeline: deskew $\rightarrow$ binarise $\rightarrow$ line/word segmentation $\rightarrow$ recognition $\rightarrow$ post-correction $\rightarrow$ layout reconstruction.
- Multi-language capability: an English model plus an optional Urdu/Arabic-script proof-of-concept model.
- Export to TXT, DOCX and searchable PDF, preserving basic line/paragraph structure.
- A desktop/web application with file/PDF upload, near-real-time webcam capture, and an editable side-by-side review panel.
- Complete documentation: code repository, training notebooks, evaluation report (CER/WER, error analysis) and user guide.

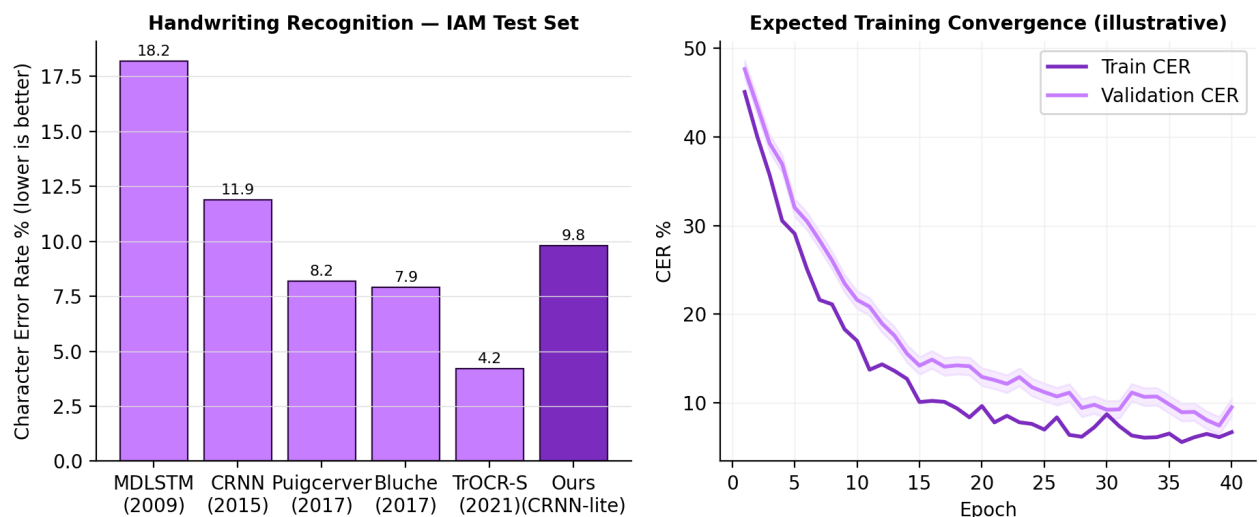


Figure 2.3: (left) Character Error Rate of representative HTR methods on the IAM test set with our expected operating point; (right) an illustrative training/validation CER convergence curve.

12. Innovation / Uniqueness

- **Complete document workflow:** Not just a recogniser — a full pipeline (pre-process → segment → recognise → correct → rebuild layout → export) wrapped in a usable app.
- **Hybrid accuracy boosting:** Combines a compact CRNN with CTC beam search and a dictionary/edit-distance post-corrector, getting near-Transformer usefulness at a fraction of the compute.
- **Multi-language design:** Pluggable model + dictionary architecture, demonstrated with English and an Urdu/Arabic-script proof-of-concept — relevant to the local context.
- **Low-resource & reproducible:** Lightweight CNN backbone, synthetic-data pre-training and free cloud GPUs make the whole project reproducible on student laptops.
- **Accessibility & preservation focus:** Editable, searchable, screen-reader-friendly output supports visually-impaired users and archival digitisation.
- **Practical export formats:** Direct export to DOCX and searchable PDF (OCR text layer) — features real users actually need, rarely included in coursework projects.

13. Feasibility Analysis

13.1 Technical Feasibility

CRNN+CTC has many open reference implementations; IAM is freely available for research; OpenCV, PyTorch, Tesseract, SymSpell, python-docx and pdf2image are mature. A compact CRNN has only a few million parameters and trains in a few hours on a free Colab T4 GPU, especially when warm-started from synthetic-data pre-training. Inference on a single text line is millisecond-scale on CPU, so the application runs comfortably on student laptops.

13.2 Cost Analysis

Item	Option Chosen	Cost (PKR)
------	---------------	------------

Compute for training	Google Colab / Kaggle free GPU tier	0
Datasets	IAM (free for research) + synthetic data we generate	0
Software & libraries	Python, PyTorch, OpenCV, Tesseract, SymSpell, python-docx (open-source)	0
Development machines	Members' existing laptops	0 (already owned)
Optional Colab Pro / Kaggle add-on	Only if final tuning needs more GPU time	≈ 2 800 / month (optional)
Demo hosting (optional)	Streamlit Community Cloud / Hugging Face Spaces (free)	0
Estimated total	Core project	≈ 0 (PKR 0–3 000 if Colab Pro used)

Table 2.4: Indicative cost analysis — effectively zero-budget.

13.3 Hardware Feasibility & GPU / Resource Limitations

- Training: a free Colab/Kaggle GPU (T4/P100) is sufficient for a compact CRNN; full TrOCR-scale Transformer training is intentionally out of scope.
- Local development/inference: any laptop with 8 GB+ RAM; no GPU strictly required for inference.
- Mitigations for limited resources: synthetic-data pre-training to converge faster, fixed small line-image height to bound memory, gradient clipping and mixed precision, and checkpoint-and-resume to survive Colab session limits.

13.4 Practical Implementation Plan

Weeks 1–4: literature, IAM download, pre-processing/segmentation pipeline, synthetic-data generator, per-character CNN baseline. Weeks 5–8: build and train the CRNN+CTC model on Colab, reach target CER, add beam-search decoding. Weeks 9–12: SymSpell post-correction, multi-language model, layout reconstruction, export module, UI. Weeks 13–16: integration, testing on custom handwriting, evaluation report, documentation and final demo. Roles: pre-processing & segmentation (member A), model & training (members A & B), post-processing & multi-language (member C), UI/export & documentation (member D), with weekly integration reviews.

14. Hardware and Software Requirements

Requirement	Minimum	Recommended
Processor	Quad-core CPU (Intel i5 / Ryzen 5)	8-core CPU
RAM	8 GB	16 GB
GPU (local, optional)	Integrated GPU (CPU inference fine)	NVIDIA GPU 4 GB+ (e.g. GTX 1650)
GPU (training)	Google Colab / Kaggle free T4	Colab Pro / institutional GPU

Storage	15 GB free	30 GB free (IAM + synthetic data + checkpoints)
OS	Windows 10 / Ubuntu 20.04 / macOS	Ubuntu 22.04
Software	Python 3.10, PyTorch, OpenCV, Tesseract, SymSpell, python-docx, Streamlit, Git	+ CUDA toolkit (if local GPU), VS Code
Peripherals	Scanner or smartphone camera; optional webcam	Flatbed scanner for clean page captures

Table 2.5: Hardware and software requirements.

15. Future Enhancements

- Fine-tune a Transformer recogniser (TrOCR-small) for higher accuracy when more compute is available.
- Full-page layout analysis: multi-column text, tables, forms, marginalia, mixed print/handwriting.
- More languages and scripts (Sindhi, Pashto, Persian) and handwritten-mathematics recognition.
- On-device mobile app (TFLite/ONNX) for instant note capture and digitisation.
- Writer adaptation / few-shot personalisation to a specific user's handwriting.
- Integration with note-taking and document-management systems; batch-folder processing.
- Active-learning loop where user corrections are fed back to improve the model.

16. Timeline / Gantt Chart

The project runs over a $\approx$ 16-week semester in three phases — foundation, modelling & development, and finalisation — with the weekly milestones below.

Project Timeline / Gantt Chart — Handwritten Text Recognition

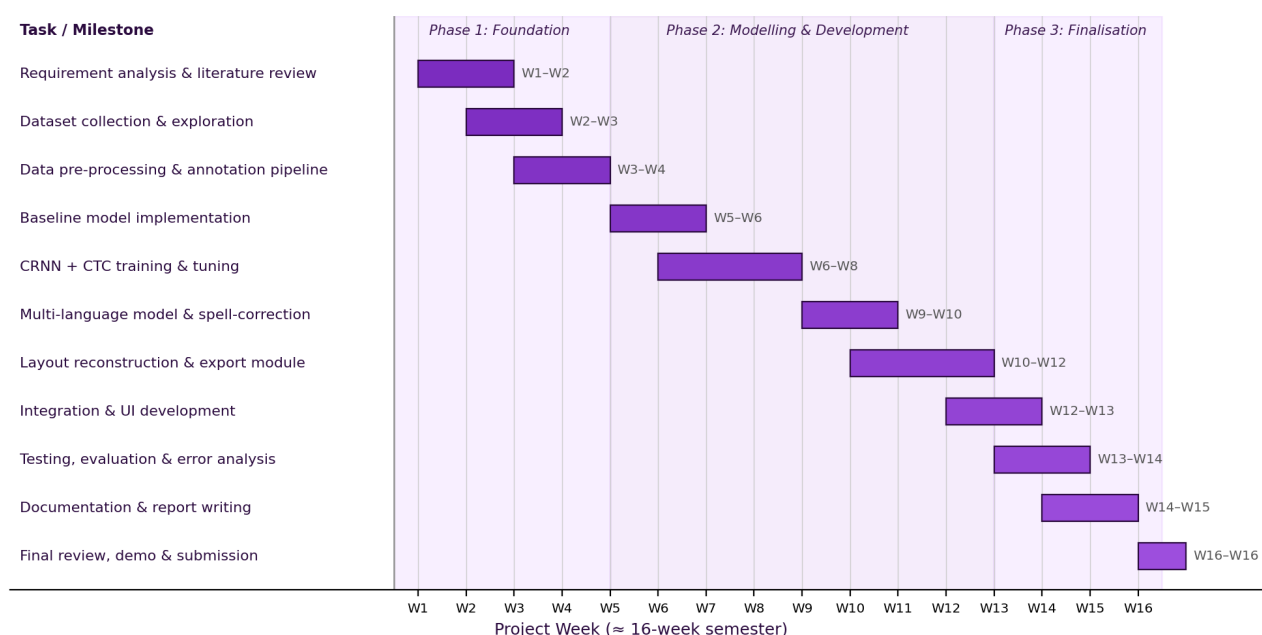


Figure 2.4: Project Gantt chart with weekly milestone breakdown.

Week(s)	Milestone / Deliverable
W1-W2	Requirement analysis, literature review, finalise CRNN architecture and tools
W2-W3	Download & prepare IAM database; exploratory analysis of lines/writers
W3-W4	Pre-processing & line/word segmentation pipeline; synthetic-data generator
W5-W6	Per-character CNN baseline; set up Colab training; build CRNN skeleton
W6-W8	Train CRNN+CTC (pre-train on synthetic, fine-tune on IAM); reach target CER; add beam search
W9-W10	SymSpell post-correction; optional Urdu/Arabic-script model proof-of-concept
W10-W12	Layout reconstruction; export to TXT/DOCX/searchable PDF; build the UI
W12-W13	End-to-end integration; webcam capture mode; performance tuning
W13-W14	Testing on IAM + custom handwriting; CER/WER evaluation and error analysis
W14-W15	Documentation, user guide and final report writing
W16	Final review, demonstration and submission

Table 2.6: Weekly milestone breakdown.

17. Conclusion

This proposal presented an AI-Based Handwritten Text Recognition system that turns images, scans and PDFs of handwriting into editable, searchable text exportable as TXT, DOCX or searchable PDF. Its recognition core is the well-proven CRNN + CTC architecture, kept deliberately compact so it can be pre-trained on synthetic data and fine-tuned on the IAM database using only a free cloud GPU. The project's value lies in delivering the complete document workflow — pre-processing, segmentation, post-correction, layout reconstruction, multi-language support and practical exports — wrapped in a usable application. It is realistic for a four-member team to complete in one semester, builds core competencies in CNNs, sequence modelling and CTC, and yields a tool with genuine utility for students, offices and archives, with a clear path toward Transformer-based and mobile future versions.

18. References

References are listed in IEEE citation style; arXiv / dataset URLs are provided where applicable.

- [1] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," IEEE Trans. Pattern

- Analysis and Machine Intelligence, vol. 39, no. 11, pp. 2298–2304, 2017. [Online]. Available: <https://arxiv.org/abs/1507.05717>
- [2]** A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 31, no. 5, pp. 855–868, 2009.
- [3]** A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376.
- [4]** J. Puigcerver, "Are multidimensional recurrent layers really necessary for handwritten text recognition?," in *Proc. IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, 2017, pp. 67–72.
- [5]** T. Bluche and R. Messina, "Gated convolutional recurrent neural networks for multilingual handwriting recognition," in *Proc. IAPR ICDAR*, 2017, pp. 646–651.
- [6]** M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in *Proc. AAAI Conf. Artificial Intelligence*, 2023. [Online]. Available: <https://arxiv.org/abs/2109.10282>
- [7]** U.-V. Marti and H. Bunke, "The IAM-database: An English sentence database for offline handwriting recognition," *Int. Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.
- [8]** A. Graves, "Supervised Sequence Labelling with Recurrent Neural Networks," *Studies in Computational Intelligence*, vol. 385, Springer, 2012.
- [9]** R. Smith, "An overview of the Tesseract OCR engine," in *Proc. IAPR ICDAR*, 2007, pp. 629–633.
- [10]** A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *NeurIPS*, 2019, pp. 8024–8035.
- [11]** G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [12]** W. Garbe, "SymSpell: 1 million times faster spelling correction & fuzzy search through Symmetric Delete spelling correction algorithm," 2012. [Online]. Available: <https://github.com/wolfgarbe/SymSpell>
- [13]** Research Group on Computer Vision and Artificial Intelligence, University of Bern, "IAM Handwriting Database." [Online]. Available: <https://fki.tic.heia-fr.ch/databases/iam-handwriting-database>
- [14]** Y. Du et al., "PP-OCR: A practical ultra lightweight OCR system," *arXiv:2009.09941*, 2020.



FINAL PROJECT PROPOSAL

Sign Language Recognition System

Proposal 3 of 3 — A Final-Year Machine-Learning Project Proposal

Riphah International University

Department of RISE MSAI

Course / Subject: Machine Learning

Submitted By — Group Members	Submitted To
1. Azhar Hussain (Group Lead) 2. Rahma 3. Haris Javaid 4. Syed Farakh Abbas	Junaid Khan Course Instructor — Machine Learning

Date of Submission: 11 May 2026

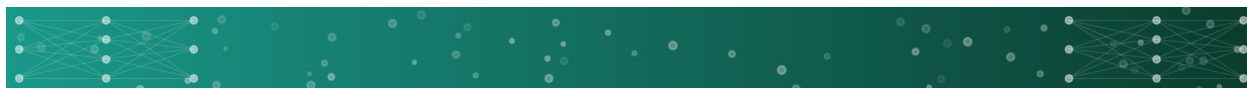
Academic Session 2025-2026 | Confidential — For Academic Evaluation Only

3.0. Contents — Proposal 3: Sign Language Recognition System

This proposal contains the standard sections (Abstract through References) as listed in the Master Table of Contents at the front of this document.

2. Abstract

Deaf and hard-of-hearing people communicate primarily through sign language, but most hearing people do not understand it, creating a daily communication barrier in schools, hospitals, banks and public offices. This project proposes a Sign Language Recognition System that uses a webcam and computer vision to translate hand-sign gestures into on-screen text and synthesised speech in real time. Rather than feeding raw images to a heavy CNN, the system uses Google's MediaPipe Hands to extract 21 three-dimensional hand landmarks per frame; these compact, lighting- and background-robust coordinates are normalised and fed to a lightweight classifier — a small multi-layer perceptron or 1-D CNN for static signs (e.g. the ASL alphabet), and an LSTM/GRU over a short frame window for dynamic gestures (motion-based words). Recognised letters/words are assembled into sentences with an auto-complete helper and spoken aloud via an offline text-to-speech engine. Because MediaPipe is highly optimised and the classifiers are tiny, the entire pipeline runs in real time on an ordinary laptop CPU with a basic webcam — no GPU required. Targeted performance is $\geq 95\%$ accuracy on a held-out set of static alphabet signs and a robust, low-latency live demo, delivered as a desktop application with text and voice output.



Keywords: *Sign language recognition, MediaPipe Hands, hand landmarks, gesture recognition, LSTM, assistive technology, real-time computer vision, text-to-speech, accessibility.*

3. Introduction

Sign languages are full natural languages expressed through hand shapes, movements, orientation, and facial expression. The World Health Organization estimates that over 400 million people worldwide have disabling hearing loss; in Pakistan, Pakistan Sign Language (PSL) serves a large Deaf community. Yet outside specialised settings there are very few interpreters, so routine interactions — a doctor's appointment, a bank transaction, a classroom question — often break down. An automatic system that recognises signs from a camera and renders them as text and speech can act as a always-available, low-cost interpreter for many everyday situations.

Computer-vision approaches to sign recognition have ranged from glove-based sensors and depth cameras (Kinect) to deep CNNs on raw RGB images and 3-D CNNs / I3D on video clips. Image-based CNNs are sensitive to lighting, skin tone, clothing and background, and video models are compute-heavy. A more practical route — and the one this project takes — is landmark-based recognition: a pretrained hand-tracking

model (MediaPipe Hands, based on the BlazePalm/hand-landmark networks of Zhang et al., 2020) outputs 21 normalised 3-D keypoints per detected hand at high frame rates on CPU. Classifying these low-dimensional, invariant features needs only a tiny neural network, which is fast to train, fast to run, and far more robust to nuisance variation. Dynamic, motion-based signs are handled by adding a short temporal model (LSTM/GRU) over a sliding window of landmark frames.

The scope is sized for a one-semester, four-member project: focus on the ASL/PSL fingerspelling alphabet plus a curated set of common dynamic words, build a clean real-time UI, and add sentence construction and offline speech output. The emphasis is on a robust, genuinely usable assistive prototype rather than maximising vocabulary size.

4. Problem Statement

Communication between Deaf signers and the hearing majority is obstructed by a chronic shortage of interpreters and the general public's lack of sign-language knowledge. Existing technical aids have drawbacks:

- **Hardware-heavy:** Glove/sensor solutions are expensive, intrusive and not always available.
- **Fragile / costly:** Raw-image CNN recognisers are brittle to lighting, background, skin tone and camera quality, and often need a GPU.
- **Slow:** Video (3-D CNN) models are accurate but too compute-intensive to run live on a typical laptop.
- **Incomplete:** Most demos recognise isolated letters only and stop there — they don't build words/sentences or produce speech.

Formally: given a live webcam stream, detect the signing hand(s), recognise the current static sign (e.g. an alphabet letter) or dynamic gesture (a motion-based word), assemble the recognised tokens into readable text with basic word completion, and output that text both on screen and as synthesised speech — all in real time on commodity, GPU-free hardware. This project delivers that end-to-end assistive pipeline.

5. Objectives

- **Primary objective:** Build a real-time webcam pipeline that detects hands (MediaPipe), recognises static ASL/PSL alphabet signs with $\geq 95\%$ accuracy on a held-out test set, and runs at ≥ 15 FPS on a laptop CPU.
- **Secondary objective:** Add a dynamic-gesture module (LSTM/GRU over a short landmark-frame window) for a curated set of common motion-based signs/words.
- **Secondary objective:** Implement landmark normalisation (translation/scale invariance, optional handedness normalisation) so recognition is robust to position, distance and background.
- **Secondary objective:** Build a sentence constructor with debouncing/temporal smoothing and an auto-complete/word-suggestion helper to turn recognised letters into fluent text.
- **Secondary objective:** Integrate offline text-to-speech (pyttsx3) — with optional online gTTS — to speak the constructed text aloud.

- **Secondary objective:** Deliver a clean desktop UI showing the webcam feed with hand-landmark overlay, the current prediction with confidence, the building sentence, and play/clear controls.
- **Learning objective:** Demonstrate competence in real-time computer vision, feature engineering from landmarks, lightweight classifiers, sequence models (LSTM/GRU), and assistive-application design.

6. Scope of the Project

In Scope

- Real-time recognition of the static ASL/PSL fingerspelling alphabet (A-Z, allowing for J/Z motion) from a webcam.
- A dynamic-gesture module for a curated set ($\approx 10\text{--}30$) of common motion-based words (e.g. hello, thanks, yes, no, please).
- MediaPipe-based hand detection and 21-landmark extraction; landmark normalisation and feature engineering.
- Lightweight classifiers (MLP / 1-D CNN for static; LSTM/GRU for dynamic) trained on public + self-collected data.
- Sentence construction with temporal smoothing/debouncing and an auto-complete helper.
- On-screen text output plus offline text-to-speech; a desktop application UI with landmark overlay and controls.

Out of Scope (Limitations)

- Full continuous sign-language translation with grammar, facial expressions and body pose (a much larger research problem).
- Large-vocabulary word recognition (e.g. full WLASL-2000) — only a curated subset is targeted.
- Two-hand complex co-articulated signs beyond the curated set; signer-independent perfection in all conditions.
- Mobile/embedded deployment (feasible later) and speech-to-sign (the reverse direction).
- Production-grade robustness in extreme lighting, motion blur or heavy occlusion.

7. Literature Review

Sign language recognition (SLR) research spans sensor-based methods, image-based deep learning, video-based temporal models, and — increasingly — skeleton/landmark-based approaches that mirror trends in human-action recognition. The works below justify our landmark-centric, lightweight design.

7.1 Key Works

Early SLR used data gloves and accelerometers, and later depth cameras (Microsoft Kinect) to obtain hand/skeleton data; these are accurate but require special hardware. With deep learning, CNNs were applied directly to RGB hand images (e.g. ASL-alphabet classifiers), and for dynamic signs, 3-D CNNs and Inflated 3D ConvNets (I3D; Carreira & Zisserman, 2017) and CNN-LSTM hybrids modelled motion in video; Li et al. (2020) released WLASL, a large word-level ASL video benchmark, with I3D baselines. In parallel, skeleton-based action recognition (e.g. ST-GCN, Yan et al., 2018) showed

that joint coordinates alone carry strong discriminative signal. The enabling technology for our approach is MediaPipe Hands (Zhang et al., 2020): a real-time, on-device hand-tracking pipeline (palm detector + hand-landmark regressor) producing 21 3-D landmarks per hand at high frame rates on CPU/mobile. Building classifiers on these landmarks — MLPs and 1-D CNNs for static poses, LSTM/GRU or lightweight Transformers/GCNs for dynamic sequences — is now a common, efficient SLR recipe; numerous IEEE/Elsevier papers report 95–99 % accuracy on alphabet datasets with such pipelines. Text-to-speech is handled by standard engines (pyttsx3 offline; Google TTS online).

7.2 Comparison of SLR Approaches

Approach	Input / Features	Strength	Limitation / Cost	Typical Accuracy*
Sensor glove / accelerometer	Bend/flex & motion sensors	Very robust signal	Special hardware; intrusive; costly	high (but impractical)
Depth camera (Kinect) + ML	Depth + skeleton joints	3-D info; lighting-robust	Requires depth sensor	≈ 90–96 %
CNN on raw RGB hand image	Cropped hand image	Simple; no landmark library	Brittle to light/background/skin; often needs GPU	≈ 90–95 %
3-D CNN / I3D on video	Stacked RGB frames	Captures motion well (dynamic signs)	Compute-heavy; not real-time on CPU	≈ 90–96 % (WLASL subsets lower)
MediaPipe landmarks + MLP/1-D CNN ★	21×3 normalised keypoints (static)	Tiny, fast, robust, CPU real-time	Static poses only; depends on landmark quality	≈ 96–99 % (alphabet)
MediaPipe landmarks + LSTM/GRU ★	Landmark sequence over a window (dynamic)	Handles motion; still lightweight	Needs sequence data collection	≈ 95–98 % (curated words)
Skeleton GCN (ST-GCN-style)	Graph over hand/body joints	Strong for complex gestures	More complex; more data	≈ 95–98 %

*Table 3.1: SLR approaches. ★ = methods adopted (MediaPipe landmarks + lightweight static and dynamic classifiers). *Accuracies are indicative ranges reported across alphabet/curated-word datasets; they are not directly comparable across different datasets.*

7.3 Datasets & Resources

Resource	Content	Use Here
ASL Alphabet (Kaggle)	≈ 87 000 images, 29 classes (A–Z + space/del/nothing)	Train/test the static-sign classifier
Sign Language MNIST	≈ 34 600 28×28 images, 24 static letters	Quick prototyping baseline
WLASL (subset)	Word-level ASL videos (2 000 words; we use a small subset)	Optional dynamic-gesture data
Self-collected landmark data	Team members record alphabet + curated words via webcam → save MediaPipe landmarks	Domain-matched data; covers PSL signs not in public sets

MediaPipe Hands, pytttsx3/gTTS, NLTK	Pretrained hand tracker; TTS engines; word lists	Feature extraction, speech output, auto-complete
--------------------------------------	--	--

Table 3.2: Datasets and resources considered.

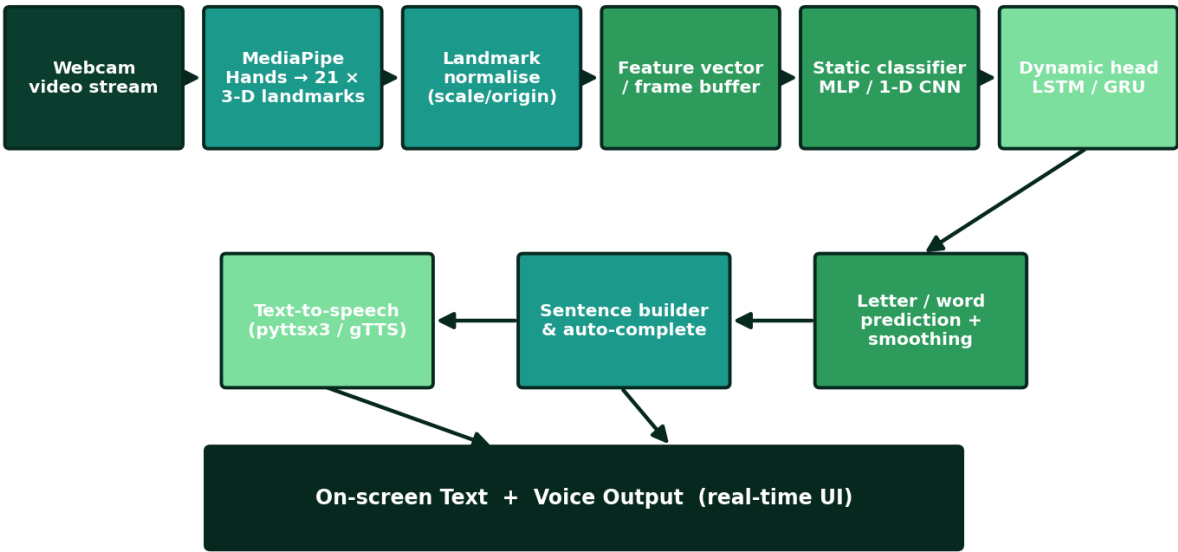
Research gap addressed: many projects classify isolated ASL letters from images and stop. Fewer deliver a robust landmark-based pipeline that also handles dynamic gestures, builds sentences with smoothing and auto-complete, speaks the output, and runs in real time on a GPU-free laptop — and fewer still consider Pakistan Sign Language. This project targets that practical, accessibility-focused gap.

8. Proposed Methodology

The methodology comprises a data-collection/feature-engineering stage, a model-training stage (separate static and dynamic models), and a real-time application/integration stage.

8.1 System Architecture

System Architecture — Sign Language Recognition



Pipeline stages flow left → right (top) then converge into the application layer (bottom).

Figure 3.1: End-to-end architecture — webcam → MediaPipe 21-landmark extraction → normalisation → static MLP/1-D CNN and dynamic LSTM/GRU classifiers → sentence builder → on-screen text + text-to-speech.

8.2 Step-by-Step Workflow

- 21. **Data collection** — Use the ASL Alphabet/Sign-Language-MNIST datasets for static signs and record additional webcam clips (team + volunteers) for PSL signs and dynamic words; for every frame, run MediaPipe Hands and store the 21×3 landmark array (plus handedness) with the label.
- 22. **Landmark normalisation & features** — Translate landmarks so the wrist is the origin; scale by a reference distance (e.g. wrist-to-middle-finger-MCP) for distance invariance; optionally mirror left hands to a canonical handedness; derive extra

- features (inter-joint distances/angles). For dynamic signs, stack normalised landmarks over a sliding window (e.g. 20-30 frames).
23. **Static classifier** — Train a small MLP (or 1-D CNN over the 63-value vector) to classify alphabet signs; use dropout and class balancing; this network has only a few thousand-tens-of-thousands of parameters and trains in minutes on CPU.
 24. **Dynamic classifier** — Train an LSTM/GRU (1-2 layers) over the landmark-sequence window to classify the curated motion-based words; pad/truncate sequences; apply temporal data augmentation.
 25. **Real-time inference** — Capture webcam frames with OpenCV; run MediaPipe; normalise; route to the static model each frame and to the dynamic model on the rolling window; obtain class + confidence.
 26. **Temporal smoothing / debouncing** — Stabilise output with a majority vote / confidence threshold over the last k frames so a letter is only 'committed' when held steady; suppress jitter and the no-hand state.
 27. **Sentence construction** — Append committed letters/words to a text buffer; map special signs to space/backspace; offer auto-complete word suggestions (frequency dictionary / prefix trie).
 28. **Text-to-speech** — On demand (or sentence-end), speak the buffer aloud using pyttsx3 (offline) or gTTS (online); show speaking status in the UI.
 29. **UI integration** — Display the webcam feed with the hand-landmark skeleton overlay, the current prediction and confidence bar, the building sentence, FPS, and buttons for speak / clear / static-vs-dynamic mode.
 30. **Testing & evaluation** — Measure per-class accuracy and a confusion matrix on held-out data, plus a live user test (recognition rate, latency, FPS) with several signers; document results.

8.3 Workflow Flowchart

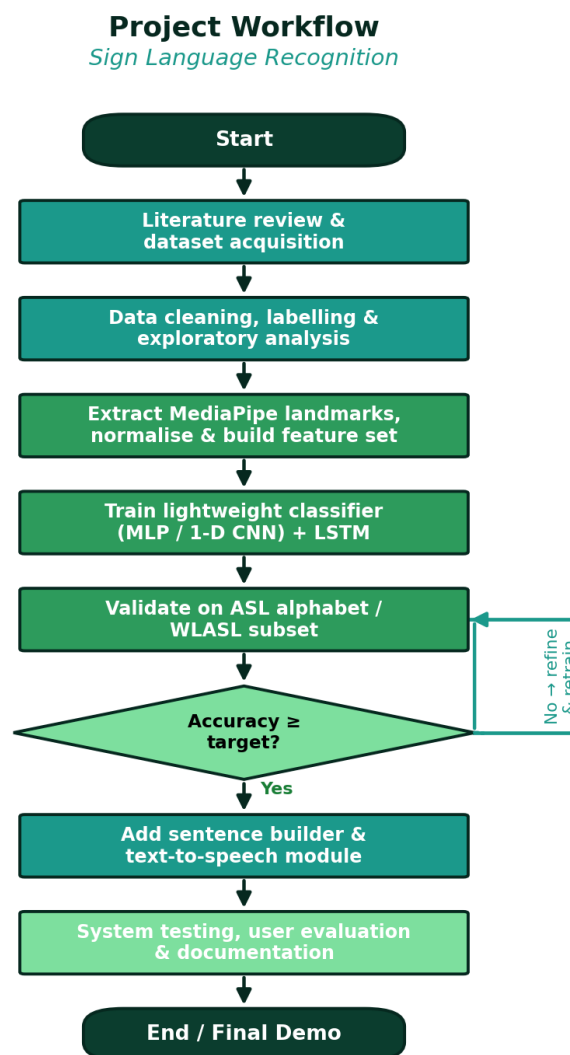


Figure 3.2: Project workflow with the accuracy decision gate and refinement loop.

8.4 Algorithms, Models & Frameworks

- **MediaPipe Hands (pretrained):** Palm detector + hand-landmark model → 21 3-D landmarks per hand, real-time on CPU.
- **Static classifier:** Small MLP (e.g. 63→128→64→#classes) or 1-D CNN over the landmark vector.
- **Dynamic classifier:** 1-2-layer LSTM/GRU over a window of normalised landmark frames (optional lightweight Transformer/GCN).
- **Smoothing:** Sliding-window majority vote + confidence thresholding (debouncing).
- **Text-to-speech:** pyttsx3 (offline, default) or gTTS + playback (online option).
- **Auto-complete:** Frequency-ranked dictionary / prefix trie for word suggestions.
- **Frameworks:** Python, OpenCV, MediaPipe, NumPy, scikit-learn, TensorFlow-Keras or PyTorch (for LSTM), pyttsx3/gTTS, Tkinter/PyQt or Streamlit.

9. Technologies and Tools

Category	Tools / Technologies	Purpose in the Project
Programming language	Python 3.10+	Vision pipeline, model training, UI and integration
Hand tracking	Google MediaPipe (Hands solution)	Pretrained real-time 21-landmark extraction on CPU
Computer vision	OpenCV	Webcam capture, frame handling, drawing landmark overlays
ML / DL frameworks	scikit-learn; TensorFlow-Keras or PyTorch	Static MLP/1-D CNN and dynamic LSTM/GRU models
Numerical	NumPy, pandas	Landmark arrays, feature engineering, dataset handling
Text-to-speech	pyttsx3 (offline); gTTS + playsound (online option)	Speaking the constructed sentences aloud
NLP helper	NLTK / wordfreq, prefix trie	Auto-complete and word suggestions
Application UI	Tkinter / PyQt (alt. Streamlit)	Live feed, prediction display, sentence panel, controls
Visualisation	Matplotlib, seaborn	Confusion matrices, accuracy curves, evaluation plots
Training environment	Local CPU; Google Colab (free) if needed	Tiny models train on CPU; Colab optional for LSTM
Dev tools	VS Code, Jupyter, Git/GitHub	Coding, experiments, version control, collaboration
Dataset sources	ASL Alphabet & Sign-Language-MNIST (Kaggle); WLASL subset; self-collected landmarks	Static training/testing; optional dynamic data; domain-matched PSL data

Table: Technology stack selected for cost-free, low-resource development.

10. Dataset Information

10.1 Candidate Datasets

- **ASL Alphabet (Kaggle):** $\approx 87\,000$ RGB images across 29 classes (A–Z plus space/delete/nothing); the main source for the static-sign classifier (we process images through MediaPipe to obtain landmarks).
- **Sign Language MNIST:** $\approx 34\,600$ grayscale 28×28 images of 24 static letters; a quick baseline/sanity-check dataset.
- **WLASL (subset):** Word-level ASL video clips; a small curated subset (common words) for the dynamic-gesture module.
- **Self-collected landmark dataset:** Team members and volunteers record the alphabet and $\approx 10\text{--}30$ dynamic words via webcam; for each frame MediaPipe

landmarks (21×3 + handedness) are saved with the label — this covers PSL signs and matches our deployment conditions.

10.2 Data Pre-processing

- Run MediaPipe Hands on every image/frame; discard frames with no confident hand detection; keep the 21×3 landmark array (and handedness) as the feature record.
- Normalise: subtract the wrist coordinate (translation invariance); divide by a reference bone length (scale invariance); optionally mirror left hands; flatten to a 63-value vector (static) or stack into a $[T \times 63]$ tensor (dynamic).
- Engineer optional extra features: pairwise fingertip distances and joint angles to help the classifier.
- Balance classes; shuffle; for dynamic data, segment continuous recordings into fixed-length windows with overlap; encode labels.

10.3 Training / Validation / Testing Split

Split	Proportion	Purpose
Training	$\approx 70 \%$	Fit the static MLP/1-D CNN and the dynamic LSTM/GRU
Validation	$\approx 15 \%$	Hyper-parameter tuning, early stopping, model selection
Testing	$\approx 15 \%$	Final per-class accuracy and confusion matrix; held-out signers where possible

Table 3.3: Data partitioning. Where enough signers are available, the split is signer-independent so that no signer appears in both training and testing.

10.4 Data Augmentation

Working in landmark space makes augmentation cheap and effective: small random rotations of the landmark cloud (in-plane and slight 3-D tilt), random scaling and translation, mild Gaussian jitter on each coordinate (sensor-noise simulation), random horizontal mirroring (handedness), and — for dynamic gestures — temporal jitter (frame dropping/duplication, speed warping, window shifting). This multiplies the effective dataset size and improves robustness to camera angle, distance and signing style without collecting more raw video.

11. Expected Outcomes

- A real-time webcam application that overlays hand landmarks and recognises static ASL/PSL alphabet signs with $\geq 95 \%$ held-out accuracy at ≥ 15 FPS on a laptop CPU.
- A dynamic-gesture module recognising a curated set of common motion-based words via an LSTM/GRU over landmark sequences.
- Stable output thanks to temporal smoothing/debouncing, with the recognised letters/words assembled into sentences.
- An auto-complete/word-suggestion helper that speeds up sentence construction.
- Offline text-to-speech (pyttsx3) that speaks the constructed text aloud; optional online gTTS.

- A clean desktop UI (live feed + landmark overlay, prediction + confidence, sentence panel, FPS, controls).
- Complete documentation: code repository, data-collection scripts, evaluation report (accuracy, confusion matrix, latency/FPS) and user guide.

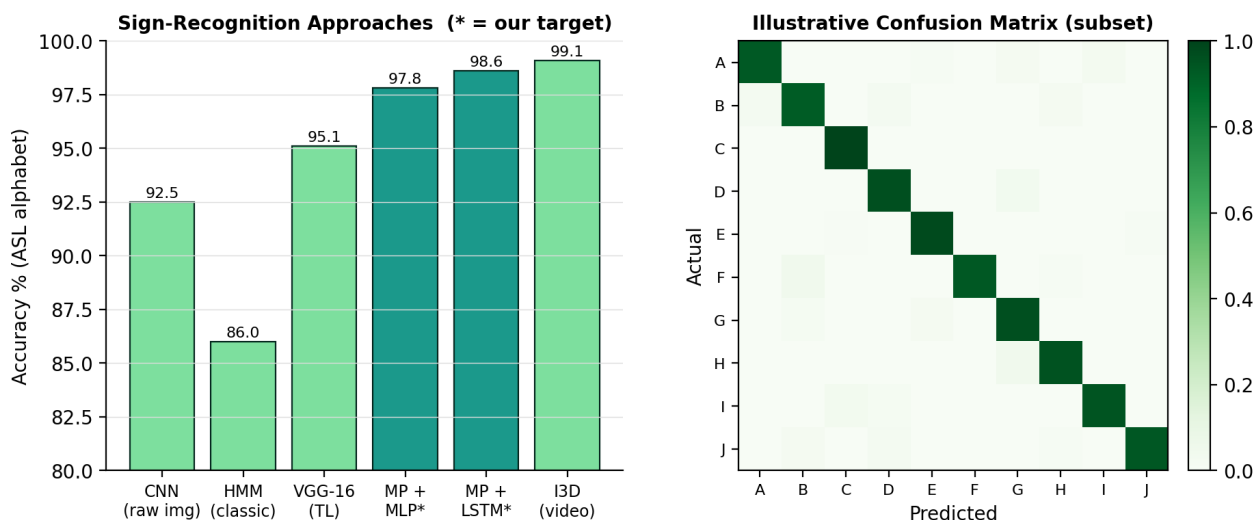


Figure 3.3: (left) Accuracy of representative sign-recognition approaches with our landmark-based targets; (right) an illustrative confusion matrix for a subset of alphabet classes.

12. Innovation / Uniqueness

- **Landmark-first, GPU-free design:** Uses pretrained MediaPipe landmarks + tiny classifiers so the whole system runs in real time on a plain laptop CPU — robust to lighting/background/skin tone.
- **Static + dynamic in one system:** Combines an instantaneous static-sign classifier with an LSTM/GRU for motion-based signs, instead of handling only isolated letters.
- **Full assistive pipeline:** Goes beyond classification to sentence building (with smoothing and auto-complete) and spoken output — a usable communication aid, not just a demo.
- **Pakistan-Sign-Language awareness:** Self-collected data includes PSL signs, addressing the local community and a gap in public datasets.
- **Reproducible & low-cost:** Free datasets and tools, tiny models trainable on CPU, optional Colab — fully reproducible by students with zero budget.
- **Robustness engineering:** Landmark normalisation (translation/scale/handedness invariance) plus landmark-space augmentation make recognition resilient to camera angle, distance and signer style.

13. Feasibility Analysis

13.1 Technical Feasibility

MediaPipe Hands is a free, well-documented, highly optimised library that already solves the hardest part (robust real-time hand landmarking). The classifiers on top are small enough to train on a CPU in minutes (static) or with a short Colab session (dynamic LSTM). OpenCV, scikit-learn/Keras, pytsx3 and Tkinter/PyQt are all mature. Numerous published projects confirm 95–99 % alphabet accuracy with this exact recipe, so the targets are realistic.

13.2 Cost Analysis

Item	Option Chosen	Cost (PKR)
Compute	Local laptop CPU (training & inference); Colab free if needed	0
Datasets	ASL Alphabet / Sign-Language-MNIST / WLASL subset (public) + self-collected	0
Software & libraries	Python, MediaPipe, OpenCV, scikit-learn/Keras, pytsx3 (open-source)	0
Camera	Built-in laptop webcam (or an inexpensive USB webcam)	0–2 500 (optional)
Development machines	Members' existing laptops	0 (already owned)
Demo packaging (optional)	PyInstaller executable / Streamlit Cloud (free)	0
Estimated total	Core project	≈ 0 (PKR 0–2 500 if a USB webcam is bought)

Table 3.4: Indicative cost analysis — essentially zero-budget.

13.3 Hardware Feasibility & GPU / Resource Limitations

- No GPU is required: MediaPipe runs in real time on CPU and the classifiers are tiny; a Colab GPU is only an optional convenience for training the dynamic LSTM.
- Any laptop with a webcam, 4–8 GB RAM and a modern dual/quad-core CPU is sufficient; FPS can be tuned by limiting MediaPipe to one hand and a moderate resolution.
- Resource mitigations: cache extracted landmarks (don't re-run MediaPipe during training), keep models small, use early stopping, and design the UI loop to skip frames gracefully under load.

13.4 Practical Implementation Plan

Weeks 1–4: literature, dataset download, MediaPipe pipeline, landmark extraction & normalisation, baseline static classifier. Weeks 5–8: collect custom data, train/tune the static MLP/1-D CNN to target accuracy, build the dynamic-data recorder. Weeks 9–12: train the dynamic LSTM/GRU, add temporal smoothing, sentence builder, auto-complete and text-to-speech. Weeks 13–16: real-time UI integration, optimisation, user testing with several signers, evaluation report, documentation and final demo. Roles: MediaPipe pipeline & features (member A), static & dynamic models (members A & B), sentence builder/TTS/auto-complete (member C), UI integration & documentation (member D), with weekly integration reviews.

14. Hardware and Software Requirements

Requirement	Minimum	Recommended
Processor	Dual/Quad-core CPU (Intel i3/i5)	Quad-core+ CPU

	/ Ryzen 3/5)	
RAM	4 GB	8 GB+
GPU	Not required	Optional NVIDIA GPU only for faster LSTM training
Camera	Built-in laptop webcam ($\geq$ 480p)	720p USB webcam
Storage	5 GB free	15 GB free (datasets + cached landmarks + models)
OS	Windows 10 / Ubuntu 20.04 / macOS	Ubuntu 22.04 / Windows 11
Software	Python 3.10, MediaPipe, OpenCV, scikit-learn/Keras, pytsx3, Tkinter/PyQt, Git	+ Jupyter, VS Code, Colab account
Audio	Working speakers/headphones (for TTS output)	—

Table 3.5: Hardware and software requirements.

15. Future Enhancements

- Expand the dynamic vocabulary toward full Pakistan Sign Language phrases; build a public PSL landmark dataset.
- Add facial-expression and body-pose cues (MediaPipe Holistic) for grammatically richer recognition.
- Move toward continuous sentence-level translation (sequence-to-sequence with a language model).
- Mobile app (MediaPipe + TFLite) for an always-available pocket interpreter.
- Two-way communication: speech-to-text plus animated/avatar sign output for the hearing-to-Deaf direction.
- Signer adaptation / few-shot personalisation; on-device continual learning from user corrections.
- Integration with video-call platforms for live captioning of signers.

16. Timeline / Gantt Chart

The project runs over a $\approx$ 16-week semester in three phases — foundation, modelling & development, and finalisation — with the weekly milestones below.

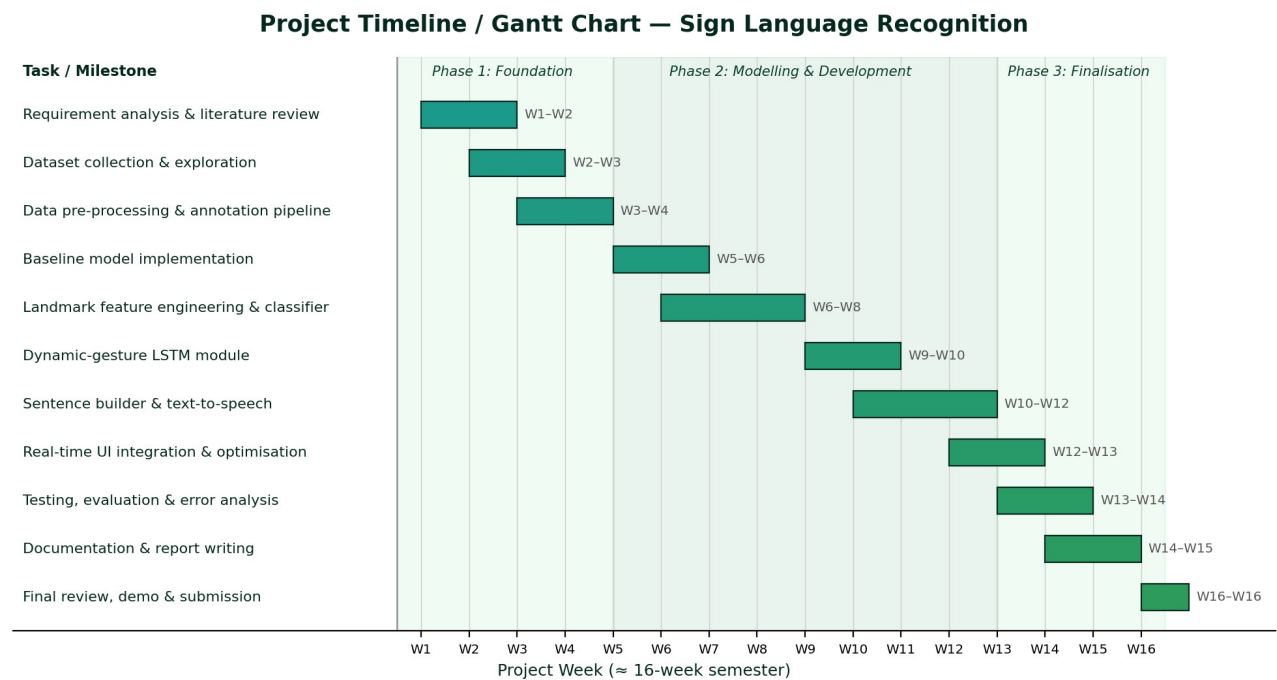


Figure 3.4: Project Gantt chart with weekly milestone breakdown.

Week(s)	Milestone / Deliverable
W1-W2	Requirement analysis, literature review, finalise architecture and tools
W2-W3	Download ASL/Sign-MNIST data; build MediaPipe landmark-extraction pipeline; EDA
W3-W4	Landmark normalisation & feature engineering; data-collection scripts; baseline static classifier
W5-W6	Collect custom alphabet/PSL data; train & tune static MLP/1-D CNN to target accuracy
W6-W8	Build dynamic-gesture recorder; assemble windowed sequence dataset; train LSTM/GRU
W9-W10	Temporal smoothing/debouncing; sentence builder with auto-complete
W10-W12	Integrate text-to-speech; build the real-time desktop UI (overlay, prediction, controls)
W12-W13	End-to-end integration; real-time optimisation (FPS, latency)
W13-W14	Testing with multiple signers; accuracy/confusion-matrix/latency evaluation
W14-W15	Documentation, user guide and final report writing
W16	Final review, demonstration and submission

Table 3.6: Weekly milestone breakdown.

17. Conclusion

This proposal presented a Sign Language Recognition System that translates webcam hand-sign gestures into on-screen text and spoken output in real time. By building on Google's pretrained MediaPipe Hands to obtain compact, robust 21-point landmarks and classifying them with tiny neural networks — an MLP/1-D CNN for static signs and an LSTM/GRU for dynamic ones — the system achieves high accuracy while running comfortably on a GPU-free laptop. Beyond classification it delivers a complete assistive pipeline: temporal smoothing, sentence construction with auto-complete, and offline text-to-speech, with optional support for Pakistan Sign Language. The project is realistic for a four-member team within one semester, builds strong skills in real-time computer vision and sequence modelling, and produces a socially valuable communication aid with a clear roadmap toward continuous translation and mobile deployment.

18. References

References are listed in IEEE citation style; arXiv / dataset URLs are provided where applicable.

- [1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," in CVPR Workshop on Computer Vision for Augmented and Virtual Reality, 2020. [Online]. Available: <https://arxiv.org/abs/2006.10214>
- [2] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.
- [3] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison (WLASL)," in Proc. IEEE Winter Conf. on Applications of Computer Vision (WACV), 2020, pp. 1459–1469.
- [4] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the Kinetics dataset (I3D)," in Proc. IEEE/CVF CVPR, 2017, pp. 6299–6308.
- [5] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition (ST-GCN)," in Proc. AAAI Conf. on Artificial Intelligence, 2018.
- [6] O. Koller, J. Forster, and H. Ney, "Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers," Computer Vision and Image Understanding, vol. 141, pp. 108–125, 2015.
- [7] N. C. Camgoz, S. Hadfield, O. Koller, H. Ney, and R. Bowden, "Neural sign language translation," in Proc. IEEE/CVF CVPR, 2018, pp. 7784–7793.
- [8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [9] IEEE Xplore, "Sign language recognition using deep learning — search results." [Online]. Available: <https://ieeexplore.ieee.org/search/searchresult.jsp?queryText=sign+language+recognition+deep+learning>
- [10] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [11] F. Chollet et al., "Keras," 2015. [Online]. Available: <https://keras.io>
- [12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [13] N. M. Bhagat (Akash) and contributors, "pyttsx3 — offline text-to-speech conversion library for Python," documentation, 2023. [Online]. Available: <https://pyttsx3.readthedocs.io>

- [14]** Akash / Kaggle, "ASL Alphabet dataset." [Online]. Available:
<https://www.kaggle.com/datasets/grassknoted/asl-alphabet>